

一：开学篇

人工智能的方法论分类：

1. 基于规则的方法：通过直接编程实现，灵感来源于人类的启发式规则。

这种方法依赖于事先定义好的规则集，根据输入数据应用这些规则进行预测或决策。基于规则的方法常用于专家系统中。

2. 基于数据的方法：

- **专家系统：**依赖于专家或统计学家根据数据创建预测或决策规则。

- **机器学习：**基于数据科学，根据数据直接做出预测或决策。

通过训练模型来自动从数据中学习规律和模式。机器学习算法可以根据输入数据进行预测或决策，而无需显式编程规则。

机器学习定义

目标是使计算机系统能够从数据中学习并自动提取模式、进行预测或做出决策，而无需明确的指令。

Herbert Simon将学习定义为系统通过经验提高性能的任何过程。

Tom Mitchell给出了机器学习更数学化的定义，他将机器学习定义为研究一种算法，该算法在某个任务T上通过经验E改善其性能P，而**无需显式编程**。一个明确定义的学习任务由<P, T, E>表示。

为什么要学习机器学习？时机成熟

- 许多基本有效和高效的算法可用。
- 大量在线数据可用。
- 大量计算资源可用。

机器学习的发展历史可以追溯到20世纪50年代。在那个时期，Samuel的跳棋程序和Selfridge的Pandemonium模型开创了机器学习的先河。20世纪60年代，神经网络中的感知机模型、模式识别和极限学习理论开始兴起。然而，Minsky和Papert证明了感知机的局限性，这导致了对符号概念归纳的研究。此时期也涌现出了Winston的arch学习器、专家系统和知识获取难题，以及Quinlan的ID3决策树算法。

20世纪80年代，决策树和规则学习得到了进一步发展，出现了基于解释的学习、学习与规划、问题解决、类比、认知架构等领域的研究。神经网络（连接主义）和反向传播算法也重新引起了关注。此时期还诞生了Valiant的PAC学习理论，研究者开始关注实验方法学。

20世纪90年代，数据挖掘、自适应软件代理和网络应用、文本学习、强化学习、归纳逻辑编程、集成学习等概念和技术逐渐兴起。支持向量机、核方法、贝叶斯网络学习等方法也得到了广泛应用。

进入21世纪，机器学习领域发展迅猛。图模型、变分推断、统计关系学习、迁移学习、序列标注、集体分类和结构化输出等技术涌现。机器学习开始在计算机系统应用领域发挥重要作用，如编译器、调试、图形处理、安全领域（入侵、病毒和蠕虫检测）、电子邮件管理、个性化助手等。机器学习也广泛应用于机器人和计算机视觉等领域。

2010年以后，深度学习成为机器学习的重要分支。大数据的学习、使用GPU或高性能计算、多任务学习和终身学习、深度强化学习等方面取得了突破。深度学习在视觉、语音、文本、网络、行为等众多领域获得了广泛应用。

学习任务的类型：

监督学习：

从带有标签的训练数据中推断出一个函数。

监督学习的目标是通过已知输入和对应的输出来学习一个模型，以便对未知的输入进行预测或分类。

监督学习的学习目标是通过给定的训练数据集（包含数据和对应的标签），让机器学习一个从数据到标签的函数。函数集合被称为假设空间，学习过程是通过更新参数使得预测结果与对应的标签尽可能接近。

在监督学习中，我们有一个包含输入数据（特征）和输出数据（标签）的训练数据集 D ，其中 x_i 表示第 i 个训练样本的输入数据， y_i 表示第 i 个训练样本的输出数据（标签）。

学习目标是使得机器学习的预测结果 $f_{\theta}(x_i)$ 与对应的标签 $y(i)$ 尽可能接近。为了衡量预测结果与标签之间的误差，引入了损失函数，它用于度量单个样本的预测与标签之间的差异。损失函数的定义取决于数据和任务的特点。

损失函数

在监督学习中，常见的损失函数包括：

- **0-1损失函数**：当预测结果与标签不一致时为1，一致时为0。
- **平方损失函数/均方误差**：度量预测值与标签之间的平方差，对离群值（预测值与真实值之间存在较大差异的样本）敏感，因为平方运算使得较大的差异被放大。因此，在使用平方损失函数时，算法会更加关注离群值的拟合。在许多回归问题中被广泛使用，例如线性回归、多项式回归等。
- **绝对损失函数**：度量预测值与标签之间的绝对差。
- **对数损失函数**：也称为对数似然损失函数，用于分类任务，度量预测的概率与真实标签之间的差异。常用于**分类问题**。
- **交叉熵损失函数**：通过计算预测概率分布与真实类别标签的差异来衡量模型的性能。其公式为 $L(Y, P) = -\sum (Y_i * \log(P_i))$ 。其中， Y 是真实类别的one-hot向量， P 是模型的预测概率分布向量， Y_i 和 P_i 分别表示真实类别和预测概率分布向量的第 i 个元素。常用于**分类问题**。
交叉熵损失函数对概率分布的形状敏感，因此在应用时需要确保模型的输出经过适当的**激活函数**（如Softmax）得到概率分布。

Loss function

0-1 loss function

$$L(Y, f(X)) = \begin{cases} 1, & Y \neq f(X) \\ 0, & Y = f(X) \end{cases}$$

quadratic loss function

$$L(Y | f(X)) = \sum_N (Y - f(X))^2$$

absolute loss function

$$L(Y, f(X)) = |Y - f(X)|$$

logarithmic loss function、log-likelihood loss function

$$L(Y, P(Y | X)) = -\log P(Y | X)$$

学习的过程就是通过调整参数（ θ ）来最小化损失函数，使得预测结果与标签的误差尽可能小。这通常通过优化算法来实现，例如梯度下降法。

参数更新

参数更新的过程可以描述为以下几个步骤：

1. 初始化参数：首先，需要对模型的参数进行初始化。参数的初始化可以是随机的，也可以根据先验知识进行设置。
2. 计算损失函数的梯度：根据当前参数的取值，计算损失函数关于参数的梯度。梯度表示了损失函数在当前参数点的变化率，指示了应该朝着哪个方向进行参数更新。
3. 参数更新：根据具体的优化需求和问题特点，选择适合的优化算法，使用梯度或其他相关信息来更新参数。更新的具体方式取决于所选算法的特点。
 - 梯度下降法：在梯度下降法中，根据梯度的反方向进行参数更新。通过乘以学习率（learning rate）来控制参数更新的步长，从而避免更新过大或过小。

- 随机梯度下降法：与梯度下降法类似，但每次更新只使用一个样本或一小批样本的梯度来估计整体梯度，从而减少计算开销。
 - 牛顿法：牛顿法利用了损失函数的二阶导数信息来进行参数更新，能够更快地收敛到最优解。
 - 其他优化算法：还有许多其他的优化算法，如共轭梯度法、拟牛顿法（如BFGS和L-BFGS）、Adam等。这些算法都有各自的特点和适用场景。
4. 迭代更新：通过重复执行步骤2至步骤3，直到达到预定的停止条件，如达到最大迭代次数、损失函数收敛等。

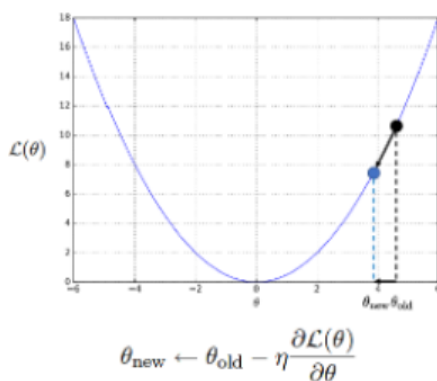
Online 问题2：如何更新参数

How to update the parameters?

Given θ , we have code that can compute

- $J(\theta)$
- $\frac{\partial J(\theta)}{\partial \theta_i}$ (for θ_i $j=0, 1, \dots, n$)

Gradient learning method:



Other Optimization algorithms:

- Conjugate gradient method
- Quasi-Newton method
 - BFGS
 - L-BFGS
- ...

正则化

L2-norm和L1-norm是常见的正则化项，用于在训练过程中对模型的参数进行约束，以避免过拟合或提高模型的稀疏性。

L2-norm（也称为岭回归或权重衰减）用于惩罚模型参数的平方和，其公式如下：

$$|\theta| * 2^2 = \sum * j = 1^n \theta_j^2$$

其中， θ 表示模型的参数， n 表示参数的数量。

L2-norm的特点是，它倾向于使模型的参数分布更加平滑，避免参数过大。通过在损失函数中加入L2-norm项，并通过调节正则化参数 λ 的大小，可以控制正则化的程度。L2正则化有助于降低模型对训练数据的过拟合风险。

L1-norm用于惩罚模型参数的绝对值之和，其公式如下：

$$|\theta| * 1 = \sum * j = 1^n |\theta_j|$$

L1-norm的特点是，它倾向于产生稀疏的参数解，即将一些参数置为零，从而使得模型更加简洁和可解释。L1正则化在特征选择和模型压缩等任务中具有重要作用。

通过在损失函数中加入L1-norm项，并通过调节正则化参数 λ 的大小，可以控制正则化的程度。L1-norm能够促使一些特征的权重变为零，从而实现特征选择和模型简化的效果。

在实际应用中，可以根据问题的性质和需求选择适当的正则化项，或者通过结合L1-norm和L2-norm的方式进行综合正则化（如Elastic Net正则化）。这些正则化技术能够提高模型的泛化能力和稳定性，并提供一定程度的参数控制和特征选择能力。

超参数

超参数 (Hyperparameters) 是关于模型的高层概念，如模型的复杂度或学习能力等。它们在标准模型训练过程中无法直接从数据中学习，需要预先定义。可以通过设置不同的值、训练不同的模型，并选择测试效果更好的值来确定超参数。

模型选择（或超参数优化）关注如何选择最优的超参数。通过尝试不同的超参数值，训练和测试不同的模型，并选择测试效果最好的超参数值，从而找到最佳的模型配置。

假设空间

一个监督学习机器可以被看作是一个探索“假设空间”(Hypothesis Space)的设备。

监督机器学习的艺术在于：

- 决定如何表示输入和输出
- 选择一个既能够表示输入和输出之间关系又简单到可以被搜索的假设空间。

不同的学习方法假设不同的假设空间（表示语言）和/或采用不同的搜索技术。

无监督学习：

尝试在没有标签的训练数据中找到隐藏的结构。 eg:clustering

无监督学习的目标是发现数据中的模式、聚类或异常值等信息，而不需要预先知道数据的类别或标签。

强化学习：

学习在动态环境中采取行动的策略，并获取奖励。

强化学习的目标是通过与环境的交互来学习最优的行为策略，使得在不同情境下获得最大的奖励或回报。

	监督学习	无监督学习
离散	分类Classification or Categorization	聚类Clustering
连续	回归Regression	降维Dimensionality Reduction

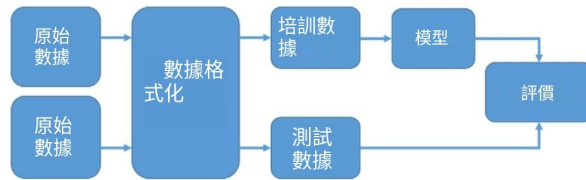
机器学习适用于：

- 构建系统过于困难或昂贵，因为需要特定的详细技能或针对特定任务的知识（如语音/人脸识别）。
- 即使我们对如何进行构建有好的想法，程序可能会非常复杂（如机器人臂、自主直升机、手写识别、大部分自然语言处理、围棋等）。
- 人类专业知识不可用（如火星导航）。
- 解决方案会随时间变化（如计算机网络上的路由）
- 开发能够自动适应和个性化定制给个体用户的系统（产品推荐算法）
- 从大型数据库中发现新知识，即数据挖掘（用户流量分析）

机器学习的一般过程：

基本假设：训练数据和测试数据中存在相同的模式

機器學習過程



1. 原始数据收集
2. 数据形式化处理
3. 训练数据准备
4. 测试数据准备
5. 模型选择和训练
6. 模型评估

以图像分类为例：

训练：给定一个带有标签的训练样本集 $\{(x^1, y^1), \dots, (x^N, y^N)\}$ ，通过最小化在训练集上的预测误差来估计预测函数 f 。函数集合 $f(x)$ 被称为假设空间，学习的过程是更新参数 θ ，使得预测值接近对应的标签值。

测试：将 f 应用于以前从未见过的测试样本 x ，并输出预测值 $y = f(x)$ 。

思考题

分别举出一个监督学习和非监督学习的例子

监督学习的例子：

分类任务：根据患者的症状和体征，预测其是否患有某种疾病。在训练阶段，算法会使用已知患有或未患有该疾病的患者的数据进行训练，然后根据这些训练数据构建模型。一旦模型训练完成，就可以对新的患者进行分类预测。

回归任务：根据房屋的特征（如面积、位置、卧室数量等），预测房屋的价格。在训练阶段，算法使用已知房屋的特征和对应的价格数据进行训练，建立一个回归模型。然后，使用这个模型对新的房屋特征进行预测，得出价格估计值。

非监督学习的例子：

聚类任务：根据客户的购买行为和偏好，将客户分为不同的市场细分。在聚类任务中，算法会根据客户的特征和相似性，将客户划分为不同的群组，以便了解不同群组的购买偏好和行为模式。

异常检测：监测网络流量中的异常活动，例如检测网络攻击或异常的数据传输行为。通过对正常网络流量进行建模，非监督学习算法能够识别出与正常模式不符的异常行为，并标识出潜在的安全风险。

二：线性回归

线性模型易于理解和实现，高效且可扩展，包括线性回归和线性分类。

模型表示

$$f_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + \theta_0$$

其中 θ_i 表示模型的参数， x_i 表示样本的特征值。特征向量表示为 $(x_1, x_2, \dots, x_n)^T$ 。

在监督学习中，给定一个包含（数据，标签）对的训练数据集。

数据集表示为 $D = (x^{(i)}, y^{(i)})_{i=1}^N$ ，其中 $x^{(i)}$ 是第 i 个训练样本的输入数据（特征）， $y^{(i)}$ 是对应的输出数据（标签）。

第 i 个训练样本的特征为 $x_i = (x_{1i}, x_{2i}, \dots, x_{ni})^T$, 对应的输出数据 (标签) 为 y_i 。

机器学习的目标是学习一个从数据到标签的函数, 即 $f_\theta(x^{(i)}) \approx y^{(i)}$ 。

$$f_\theta(x_i) = \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots + \theta_n x_n^{(i)} + \theta_0$$

函数集合 $f(\cdot)$ 被称为假设空间, 学习的过程是更新参数 θ , 使得预测值接近对应的标签值。

梯度下降

批量梯度下降 Batch Gradient Descent (**BGD**) 是一种基于批量训练样本的梯度下降算法, 用于更新模型的参数 θ 。每一次迭代中, 通过计算所有训练样本的梯度平均值来更新 θ 。这个算法会在每一次迭代中使用整个训练集, 因此每个参数更新的步长相对较慢。

随机梯度下降 Stochastic Gradient Descent (**SGD**) 是一种基于单个训练样本的梯度下降算法, 每次迭代只使用一个训练样本来更新参数 θ 。相比于BGD, SGD的学习速度更快, 但参数的更新会受到训练样本的不确定性和波动性的影响。

小批量梯度下降 Mini-Batch Gradient Descent (**MBGD**) 是介于BGD和SGD之间的一种折中方法。它每次迭代使用一小批量的训练样本来更新参数 θ , 而不是全部训练集。这样可以在一定程度上加速学习过程, 并减少参数更新的波动。

在梯度下降算法中, 学习率 α 的选择很重要。如果 α 过小, 梯度下降会很慢; 如果 α 过大, 梯度下降可能会超过最小值点, 导致无法收敛甚至发散。

为了达到收敛的条件, 通常会监测代价函数 $J(\theta)$ 的变化情况。如果 $J(\theta)$ 趋于稳定或下降到一个可接受的程度, 就可以停止迭代并得到最优的参数 θ 。

随机梯度下降

Stochastic Gradient descent

“stochastic”: Each step of gradient descent uses single training example.

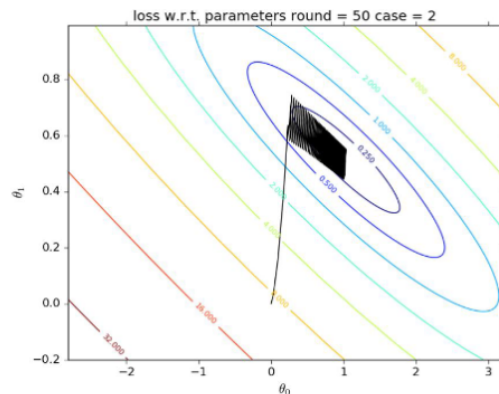
Repeat until convergence

$$\theta_0 := \theta_0 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

Compare with BGD

- Faster learning
- Uncertainty or fluctuation in learning



小批量梯度下降

Mini-Batch Gradient descent

- A combination of batch GD and stochastic GD

- Split the whole dataset into K mini-batches

$$\{1, 2, 3, \dots, K\}$$

- For each mini-batch k , perform one-step BGD toward

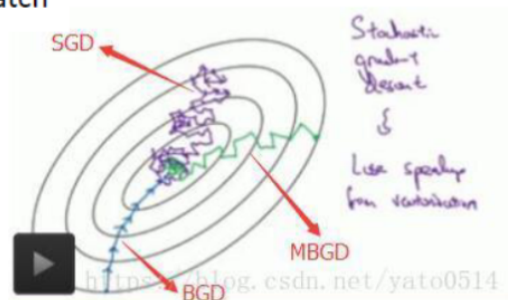
$$J^k(\theta) := \frac{1}{2N_k} \sum_{i=1}^{N_k} (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Update $\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial J^k(\theta)}{\partial \theta}$ for each mini-batch

$$\theta_0 := \theta_0 - a \frac{1}{N_k} \sum_{i=1}^{N_k} (f_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - a \frac{1}{N_k} \sum_{i=1}^{N_k} (f_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

- Good learning stability (BGD)
- Good convergence rate (SGD)



单变量线性回归

Linear regression with one variable

Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

Linear Regression Model

$$f_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

Repeat until converge

$$\begin{aligned}\theta_0 &:= \theta_0 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) \\ \theta_1 &:= \theta_1 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}\end{aligned}$$

update
 θ_0 and θ_1
simultaneously

Correct: Simultaneous update

$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 $\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 $\theta_0 := \text{temp0}$
 $\theta_1 := \text{temp1}$

Incorrect:

$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 $\theta_0 := \text{temp0}$
 $\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 $\theta_1 := \text{temp1}$

对多元线性回归求解，只需要将梯度下降的公式扩展到每一个参数即可

多变量线性回归

Linear regression with multiple variable

Hypothesis: $f_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function: $J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2$

Notation:

n = number of features

$x^{(i)}$ = input (features) of i^{th} training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$$

}

(simultaneously update for every $j = 0, \dots, n$)

新的梯度

New Gradient Descent

Previously ($n=1$):

Repeat {

$$\theta_0 := \theta_0 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) \underbrace{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$) :

Repeat {

$$\theta_j := \theta_j - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for

}

$j = 0, \dots, n$)

$$\theta_0 := \theta_0 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

...

求得最合适的参数 θ ，使代价函数最小时，梯度下降法相对于正规方程(Normal Equation)求解具有以下优势：

1. 计算效率：梯度下降法的计算复杂度较低，特别是在处理大规模数据集和高维特征时更加高效。相比之下，正规方程的计算量大，尤其当输入变量的维度较大时（ n 比较大），计算复杂度呈指数级增长，对于大规模问题可能不可行。

2. 内存占用：梯度下降法是一种迭代的优化方法，只需在每次迭代中计算并更新参数，不需要一次性在内存中存储所有数据。这使得梯度下降法能够处理大规模数据集，而正规方程需要存储整个数据集和特征矩阵，对内存要求较高。
3. 灵活性：梯度下降法适用于各种类型的模型和代价函数，而正规方程只适用于线性回归模型。梯度下降法可以应用于更广泛的机器学习问题，包括非线性回归、逻辑回归和神经网络等。
4. 局部最优解：梯度下降法可以帮助我们逃离局部最优解，并在参数空间中搜索全局最优解。正规方程求解给出的是解析解，可能会受到数据噪声和特殊分布的影响，容易陷入局部最优解。

正规方程(Normal Equation)求解方法的优点是能够一次性得到解

特征归一化

特征归一化（Feature Scaling）是一种常见的数据预处理操作，用于将不同特征的取值范围缩放到大致相同的尺度上。目的是确保不同特征的取值范围相似，避免某些特征对模型训练产生过大的影响。这有助于提高模型的收敛速度、稳定性和预测性能。

一种常见的特征归一化方法是将特征进行标准化（也称为**Z-score标准化**）。其公式表示如下：

$$\hat{x} = \frac{x - \mu}{\sigma}$$

其中， x 是原始特征值， $\hat{x}$ 是归一化后的特征值， μ 是原始特征值的均值， σ 是原始特征值的标准差。通过减去均值并除以标准差，可以使特征的均值为0，标准差为1。

另一种常见的特征归一化方法是将特征进行缩放到固定的区间范围（例如[0, 1]或[-1, 1]）。其公式表示如下：

$$\hat{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

其中， $x_{\min}$ 和 $x_{\max}$ 分别是特征的最小值和最大值。通过减去最小值并除以最大值和最小值的差，可以将特征缩放到指定的区间范围内。

学习率选择

学习率选择在梯度下降算法中起着重要的作用，可以根据每次迭代后代价函数的变化情况进行观察和调整，打印出每次迭代或每隔几次迭代后的代价函数值，观察其是否逐渐下降。如果代价函数没有适当地下降，就需要调整学习率。

学习率的选择是一个平衡的过程。如果学习率过小，梯度下降算法收敛速度会很慢；如果学习率过大，梯度下降可能会超过最小值点，导致无法收敛或发散。

学习率优化算法

- Momentum（动量）：帮助加速随机梯度下降（SGD）朝着相关方向前进，并减小震荡。其通过引入动量项来更新参数。
- Adagrad（自适应梯度）：根据参数的频率自适应地调整学习率，对于不经常更新的参数执行较大的更新，对于频繁更新的参数执行较小的更新。
- RMSProp（均方根传播）：通过将学习率除以梯度平方的指数移动平均来调整学习率，以适应不同参数的变化情况。
- Adam（自适应矩估计）：类似于RMSProp，它不仅存储梯度平方的指数移动平均，还保持过去梯度的指数移动平均，类似于动量的作用。

梯度下降和最小二乘法对比

梯度下降：

- 参数更新公式： $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$
- 需要选择学习率 α 。
- 需要进行多次迭代。
- 在特征数目较大时也能良好工作。

最小二乘法是一种解析解方法，通过对代价函数进行最小化的数学运算，直接得到参数的解析表达式。梯度下降法是一种迭代优化方法，通过不断更新参数来逐步降低代价函数的值。

最小二乘法：

我们的目标是求使 $E_{\hat{\theta}}$ 最小的 $\hat{\theta}$ 。

$$\hat{\theta}^* = \arg \min_{\hat{\theta}} (y - \hat{y})^T (y - \hat{y})$$

想得到最优解，即对 $\hat{\theta}$ 求导数得到：

$$\frac{\partial E_{\hat{\theta}}}{\partial \hat{\theta}} = 2X^T(\hat{y} - y) = 2X^T(X\hat{\theta} - y)$$

使上式为0，得到：

$$\hat{\theta}^* = (X^T X)^{-1} X^T y$$

- 参数求解公式： $\theta = (X^T X)^{-1} X^T y$
- 不需要选择学习率 α 。
- 当特征数目很大时（>10000），速度较慢。
- 不需要进行迭代。
- 需要计算 $(X^T X)^{-1}$ 。
- 仅适用于线性模型。
- 有时无法直接计算（如果 $X^T X$ 不可逆）。
- 需要进行多次迭代。
- 在特征数目较大时仍能良好工作。

当 $X^T X$ 不可逆时，考虑引入岭回归：

岭回归（Ridge Regression）是针对多元线性回归中输入变量数目大于样本点数目的情况引入的一种技术。它通过在代价函数中添加正则项来限制参数的大小，并减少不重要的参数。岭回归的代价函数为：

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2$$

其中， λ 是正则化参数，用于控制正则项的权重。

岭回归的最优解可以通过求导数为零得到：

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

其中， I 是单位矩阵。

岭回归的引入解决了输入数据矩阵非满秩的问题，通过增加一个正则项，使得矩阵变成非奇异矩阵，从而可逆。同时，正则项对参数施加惩罚，避免某个输入变量的参数过大导致模型受其主导的情况发生。

总结

- 梯度下降适用于大量特征的情况，需选择学习率，需要进行多次迭代。
- 最小二乘法适用于线性模型，特征数目较小，满秩矩阵情况，无需迭代，无需选择学习率，计算矩阵逆。
- 岭回归通过增加正则项解决了输入数据矩阵非满秩的问题，限制参数的大小，避免某个参数过大对模型的影响。

评价标准

MSE(Mean Squared Error)
均方误差

$$\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f(x^{(i)}))^2$$

RMSE (Root Mean Squared Error)
均方根误差

$$\sqrt{\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f(x^{(i)}))^2}$$

MAE (Mean absolute Error)
平均绝对误差

$$\frac{1}{N} \sum_{i=1}^N |(y^{(i)} - f(x^{(i)}))|$$

R-Squared (R2score) 决定系数

$$\begin{aligned} &= \frac{\sum_{i=1}^N (f(x^{(i)}) - \bar{y})^2}{\sum_{i=1}^N (y^{(i)} - \bar{y})^2} \\ &= 1 - \frac{\sum_{i=1}^N (f(x^{(i)}) - y^{(i)})^2}{\sum_{i=1}^N (y^{(i)} - \bar{y})^2} \\ &= 1 - \frac{MSE}{Var} \end{aligned}$$

思考题

多变量线性回归相比单变量回归，可能会出现哪些问题？

1. **维度灾难 (Curse of Dimensionality)**：随着变量数量的增加，数据空间的维度也增加，导致需要更多的数据来准确表示整个数据空间。在多变量回归中，数据点的分布在高维空间中变得稀疏，这会增加模型的复杂性和泛化能力的挑战。
2. **共线性 (Collinearity)**：当多个输入变量之间存在高度相关性时，称为共线性。共线性会导致模型估计不准确、参数不稳定，以及降低模型的解释能力。共线性还使得模型难以确定每个变量对目标变量的独立贡献。
3. **过拟合 (Overfitting)**：在多变量回归中，模型可能更容易过拟合数据。过多的变量可以使模型更加灵活，可以更好地拟合训练数据，但也增加了模型在未见过的数据上的泛化误差。选择合适的变量子集或采用正则化方法可以缓解过拟合问题。
4. **特征选择与维度降低**：多变量回归中需要选择合适的特征来构建模型，但在大量变量的情况下，特征选择变得更加困难。特征选择的挑战包括确定哪些变量对目标变量有重要影响，以及如何处理高维数据。维度降低方法如主成分分析 (PCA) 可以用于减少特征数量并保留最重要的信息。
5. **数据要求更高**：多变量回归通常需要更多的数据来进行准确建模。由于数据点的增加，收集、处理和分析数据的复杂度也增加。此外，多变量回归还需要考虑数据中的缺失值、异常值和数据预处理的挑战。
6. **解释和可解释性**：多变量线性回归模型可能更难解释和理解，因为多个输入变量之间的关系可能相互影响。解释模型对每个变量的影响和权重也可能更加复杂。

综上所述，多变量回归相比单变量回归更加复杂，需要处理高维数据、共线性和过拟合等问题。适当的特征选择、维度降低和正则化方法可以帮助应对这些挑战，并提高模型的性能和泛化能力。

当回归模型出现欠拟合时，可以采取什么方法？

欠拟合（高偏差）的表现：

- 在训练数据和测试数据上表现都不理想。
- 模型过于简单，无法捕捉到数据的复杂关系。

修复方法：

- 增加训练样本数量。
- 增加特征数量，引入更多相关特征。
- 增加模型的复杂度，例如增加隐藏层的节点数或增加模型的深度。

- 选择更复杂的模型，例如使用更高级的算法或深度学习模型。

当回归模型出现过拟合时，可以采取什么方法？

模型过拟合（高方差）的表现：

- 在训练数据上表现良好，但在测试数据上表现较差。
- 模型过于复杂，过度拟合了训练数据的细节。

修复方法：

- 增加训练样本数量。
- 减少使用的特征数量。
- 调整模型复杂度，例如减少隐藏层的节点数或减小模型的深度。
- 正则化（增加正则化参数 λ ）。

三：线性分类

线性回归 + 激活函数 + 降维 = 线性分类

阶跃函数（Step Function）和S函数（Sigmoid Function）是常用于分类任务的函数。

阶跃函数

阶跃函数(Step Function)将输入映射为0或1，用于表示Negative Class和Positive Class

- 表示为：
$$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ 1, & \text{if } x \geq 0 \end{cases}$$
- 通常用于二元分类问题，其中0表示“Negative Class”（例如坏瓜），1表示“Positive Class”（例如好瓜）。

S函数

S函数(Sigmoid Function)将实数映射到0到1之间的概率值，用于表示在给定输入x的条件下，y=1的概率

- 表示为：
$$f(x) = \frac{1}{1+e^{-\theta^T x}}$$
- 也称为**逻辑函数 (Logistic Function)**。
- 在二元分类任务中，可以使用S函数来估计给定输入x时，y=1的概率，由参数 θ 进行参数化。
- 输出的解释是估计的概率，即估计在给定x的条件下，y=1的概率。

解释输出：

- 对于S函数， $f(\theta^T x)$ 可以解释为在给定x的条件下，y=1的概率。
- 对于二元分类任务，输出可以解释为：
 - $P(y = 1|x) = \frac{1}{1+e^{-\theta^T x}}$ ：在给定x的条件下，y=1的概率。
 - $P(y = 0|x) = 1 - P(y = 1|x) = \frac{e^{-\theta^T x}}{1+e^{-\theta^T x}}$ ：在给定x的条件下，y=0的概率。

逻辑斯蒂回归

逻辑斯蒂回归 (Logistic Regression) ,也称对数几率回归，是一种用于分类问题的算法，它使用对数几率 (log odds) 函数将输入映射到一个介于0和1之间的概率值。以下是对数几率回归的相关表达式和解释：

对数几率 (Log Odds)：

- 表示为：
$$\log \left(\frac{P(y=1|x)}{P(y=0|x)} \right) = \log \left(\frac{1}{1+\exp(-\theta^T x)} \right) = \theta^T x$$

其中， $P(y = 1|x)$ 表示给定输入 x 时目标变量 y 为正类的概率， $P(y = 0|x)$ 表示 y 为负类的概率， θ 是模型的参数向量。

解释对数几率输出：

- 对数几率输出表示为 $\log \left(\frac{P(y=1|x)}{P(y=0|x)} \right)$ 。

- 对于二元分类问题，可以将输出解释为给定输入 x ，目标变量 y 为正类（例如肿瘤为恶性）的概率与 y 为负类（例如肿瘤为良性）的概率的比值的对数。

实际应用：

- 假设对于肿瘤大小进行二元分类，预测肿瘤是否为恶性（Malignant）。若预测输出 $\theta^T x = 0.7$ ，则可以告诉患者肿瘤为恶性的概率为70%。
- 在二元分类中，通常将分类器的输出阈值设为0.5，即当预测概率大于0.5时，预测为正类（例如肿瘤为恶性）。

Logistic分布是一种连续型的概率分布，其**分布函数**和**密度函数**分别如下（ μ 表示位置参数， γ 表示形状参数）：

$$F(x) = P(X \leq x) = \frac{1}{1 + e^{-(x-\mu)/\gamma}}$$

$$f(x) = F'(X \leq x) = \frac{e^{-(x-\mu)/\gamma}}{\gamma(1 + e^{-(x-\mu)/\gamma})^2}$$

Logistic分布函数在 $\mu = 0, \gamma = 1$ 的时候就是sigmoid函数

非凸代价函数

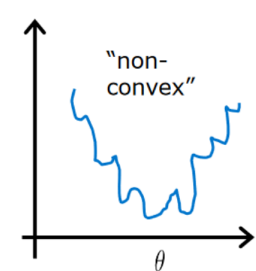
Non-convex Cost function

Linear regression:

Loss function $L(y^{(i)}, f_{\theta}(x^{(i)})) = \frac{1}{2} (y^{(i)} - f_{\theta}(x^{(i)}))^2$

Cost function $J(\theta) = \frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2$

Gradient Descent



$f_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$

$$\theta_j := \theta_j - \alpha \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

逻辑斯蒂回归使用**梯度下降**方法来更新参数。参数更新的规则为：

$$\theta_j := \theta_j - \alpha \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x_i) - y^{(i)}) x_j^{(i)}.$$

其中， $f_{\theta}(x)$ 表示模型的预测结果， $y^{(i)}$ 是第 i 个样本的标签， $x^{(i)}$ 是第 i 个样本的特征向量， N 是训练样本的数量， α 是学习率。

逻辑斯蒂回归中的损失函数/代价函数：

逻辑斯蒂回归中代价函数

Cost function in Logistic regression

$$\begin{aligned} Loss(f_{\theta}(x), y) &= \begin{cases} -\log(f_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - f_{\theta}(x)) & \text{if } y = 0 \end{cases} \\ &= -y \log(f_{\theta}(x)) - (1 - y) \log(1 - f_{\theta}(x)) \end{aligned}$$

Cost Function: $J(\theta) = \frac{1}{N} \sum_{i=1}^N L(y^{(i)}, f_{\theta}(x^{(i)}))$

$$= \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \log(f_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\theta}(x^{(i)})) \right]$$

学习目标是找到 θ ，使得代价函数最小。

逻辑斯蒂回归梯度推导

Derivation of logistic gradient

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \log(f_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\theta}(x^{(i)})) \right]$$

$$\begin{aligned} &\downarrow \\ &y^{(i)} \log\left(\frac{1}{1 + e^{-\theta^T x^{(i)}}}\right) + (1 - y^{(i)}) \log\left(1 - \frac{1}{1 + e^{-\theta^T x^{(i)}}}\right) \\ &= -y^{(i)} \log(1 + e^{-\theta^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{\theta^T x^{(i)}}) \end{aligned}$$

逻辑斯蒂回归梯度推导

Logistic gradient derivation

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \left(-\frac{1}{N} \sum_{i=1}^N (-y^{(i)} \log(1 + e^{-\theta^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{\theta^T x^{(i)}})) \right) \\ &= -\frac{1}{N} \sum_{i=1}^N \left(-y^{(i)} \frac{-x_j^{(i)} e^{-\theta^T x^{(i)}}}{1 + e^{-\theta^T x^{(i)}}} - (1 - y^{(i)}) \frac{x_j^{(i)} e^{\theta^T x^{(i)}}}{1 + e^{\theta^T x^{(i)}}} \right) \\ &= -\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f(x^{(i)})) x_j^{(i)} \\ &= \frac{1}{N} \sum_{i=1}^N (f(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

$\frac{dx^T A}{dx} = A$

逻辑斯蒂回归求解

Logistic regression solution

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \log(f_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\theta}(x^{(i)})) \right]$$

Want $\{ \min_{\theta} J(\theta) \}$:

Repeat

$$\theta_j := \theta_j - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

}

Algorithm looks identical to linear regression!

线性判别分析

线性判别分析 (Linear Discriminant Analysis) 是一种用于多类分类和特征降维的监督学习方法。它寻找一个投影子空间, 使得同一类别的样本投影尽可能接近, 同时不同类别之间的投影均值尽可能远离。以下是线性判别分析的关键要点:

1. 通过计算类内散布矩阵 (Within-class scatter matrix) 和类间散布矩阵 (Between-class scatter matrix) 来寻找最佳投影方向。其中类内散布矩阵衡量了同一类别内样本的散布情况, 类间散布矩阵衡量了不同类别之间样本的差异。
2. 线性判别分析的目标是最大化广义瑞利商 (generalized Rayleigh quotient)。通过求解广义瑞利商的最大特征值对应的特征向量, 得到最佳投影方向。
3. 在多类线性判别分析 (Multi-Class LDA) 中, 可以使用全局散布矩阵 (Global scatter matrix)、类内散布矩阵和类间散布矩阵来计算投影方向。最佳投影方向是全局散布矩阵的特征向量。
4. **对于多类分类问题, 可以将二分类方法进行扩展。** 常见的方法包括一对一 (One vs. One) 和一对其余 (One vs. Rest) 策略, 以及多对多 (Many vs. Many) 策略。

线性判别分析的数学公式如下:

1. 类内散布矩阵 (Within-class scatter matrix) : $S_w = \sum_{j=1}^m S_{w_j}, \quad S_{w_j} = \sum_{x \in X_j} (x - \mu_j)(x - \mu_j)^T$
2. 类间散布矩阵 (Between-class scatter matrix) : $S_b = S_t - S_w, \quad S_t = \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T$
3. 广义瑞利商 (Generalized Rayleigh quotient) : $W^* = \arg \max_W \frac{W^T S_b W}{W^T S_w W}$

多类分类问题

多个二分类器解决多分类问题 (Binary classifiers for multiple classification) :

1. 一对一策略 (One-vs-One) :
 - 分割阶段: 将N个类别分割成两两组合, 生成N(N-1)/2个二分类器。

- 测试阶段：将新样本输入所有分类器进行预测，得到N(N-1)/2个分类结果，通过投票得到最终的多分类结果。
2. 一对其余策略（One-vs-Rest）：
- 分割阶段：选择一个类别作为正例，其他类别作为负例，生成N个二分类器。
 - 测试阶段：将新样本输入所有分类器进行预测，得到N个分类结果，比较每个分类器的预测置信度。

在这些策略中，常常会遇到以下问题：

1. 类别不平衡（Class imbalance）：两个类别的训练样本数量差异巨大。可以通过欠采样或过采样来处理类别不平衡的问题。称为重新平衡（Rescaling）。
 - 欠采样（Undersampling）：减少样本数量，使得多数类别和少数类别的样本数量接近。
 - EasyEnsemble: 通过随机欠采样生成多个训练子集，每个子集都包含多数类别的一部分样本和全部的少数类别样本。
 - BalanceCascade: 通过反复训练和欠采样生成多个分类器，每次欠采样都基于前一次分类器的错误预测样本。
 - 过采样（Oversampling）：增加样本数量，使得多数类别和少数类别的样本数量接近。
 - SMOTE（Synthetic Minority Over-sampling Technique）：基于少数类别样本，通过合成新的样本来增加少数类别样本数量。
 - Borderline SMOTE: 在SMOTE的基础上，仅合成边界附近的样本，以更好地保持决策边界。
 - ADASYN（Adaptive Synthetic Sampling）：根据样本密度分布，在密度较低的区域合成更多的少数类别样本。
2. 阈值调整（Threshold-moving）：根据分类器的输出概率调整分类的阈值，以达到更好的性能。

Softmax回归

Softmax回归（Softmax Regression）是一种常用的多类分类方法，通过概率分布实现分类。它使用Softmax函数将样本的特征与权重进行线性组合，并转化为概率分布。Softmax回归的目标是最小化损失函数，可以使用梯度下降等优化算法进行求解。

1. Softmax函数： $\sigma(z) * j = \frac{e^{z_j}}{\sum_{k=1}^m e^{z_k}}$
2. Softmax回归模型预测： $J(\theta) = \min_{\theta} -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^m \mathbb{1}y^{(i)} = j \log \left(\frac{e^{\theta_j^T x^{(i)}}}{\sum_{k=1}^m e^{\theta_k^T x^{(i)}}} \right)$
3. 梯度下降更新权重： $\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{N} \sum_{i=1}^N (\sigma(\theta_j^T x^{(i)}) - \mathbb{1}y^{(i)} = j) x^{(i)}$

性能评价-混淆矩阵

分类性能评价中的混淆矩阵（Confusion matrix）是一种用于可视化分类器性能的工具，它总结了分类结果的真实标签与预测标签之间的关系。混淆矩阵以四个指标为基础：真正例（True Positive, TP）、真负例（True Negative, TN）、假正例（False Positive, FP）和假负例（False Negative, FN）。

分类性能评价：真正率和假正率
True Positive Rate & False Positive Rate

Label	Prediction	
	1	0
1	True Positive	False Negative
0	False Positive	True Negative

- True / False
 - True: prediction = label
 - False: prediction ≠ label
- Positive / Negative
 - Positive: predict y = 1
 - Negative: predict y = 0
- True Positive Rate: $TPR = \frac{TP}{TP + FN}$
- False Positive Rate: $FPR = \frac{FP}{FP + TN}$

混淆矩阵的形式如下：

	预测为正例	预测为负例
实际为正例	True Positive	False Negative

	预测为正面	预测为负面
实际为负面	False Positive	True Negative

根据混淆矩阵可以计算出以下分类性能评价指标：

1. 精度 (Accuracy) 和错误率 (Error) :

- 精度：预测正确的样本数量与总样本数量之比，即 $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ 。
- 错误率：预测错误的样本数量与总样本数量之比，即 $Error = \frac{FP+FN}{TP+TN+FP+FN}$ 。

2. 真正率 (True Positive Rate, 也称为灵敏度、召回率) 和假正率 (False Positive Rate) :

- 真正率：真正例的数量与实际正例的数量之比，即 $TPR = \frac{TP}{TP+FN}$ 。
- 假正率：假正例的数量与实际负例的数量之比，即 $FPR = \frac{FP}{FP+TN}$ 。

3. 查准率 (Precision) 和查全率 (Recall, 也称为灵敏度、真正例率) :

- 查准率：真正例的数量与预测正例的数量之比，即 $Precision = \frac{TP}{TP+FP}$ 。
- 查全率：真正例的数量与实际正例的数量之比，即 $Recall = \frac{TP}{TP+FN}$ 。

4. F_1 度量 (F_1 -score) 和 F_β 度量 (F -beta score) :

- F_1 分数：综合考虑查准率和查全率的综合度量，即 $F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$ 。
- F_β 分数：在 F_1 分数的基础上引入了权重系数 β ，用于调整查准率和查全率的相对重要性，即 $F_\beta = (1 + \beta^2) \times \frac{Precision \times Recall}{(\beta^2 \times Precision) + Recall}$ 。

5. ROC曲线 (Receiver Operating Characteristic curve) 和AUC (Area Under ROC Curve) :

- ROC曲线：以假正率 (FPR) 为横轴，真正率 (TPR) 为纵轴绘制的曲线，用于评估分类器在不同阈值下的性能。
- AUC：ROC曲线下的面积，代表了分类器在所有可能阈值下的平均性能。AUC的取值范围在0.5到1之间，值越接近1表示分类器性能越好。

正则化

正则化。保留所有的特征，但是减少参数的大小 (magnitude)，防止过拟合的发生。

思考题

推导题：逻辑斯蒂回归中加入L2正则化，原来的代价函数和梯度下降式要怎么修改？

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \log(f_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\theta}(x^{(i)})) \right]$$

Want $\{ \min_{\theta} J(\theta) \}$:

Repeat

$$\theta_j := \theta_j - a \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

原始的逻辑斯蒂回归代价函数（Cost Function）和梯度下降式（Gradient Descent）如下所示：

$$\text{代价函数: } J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

$$\text{梯度下降式: } \theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

其中， m 是样本数量， $x^{(i)}$ 是第 i 个样本的特征向量， $y^{(i)}$ 是第 i 个样本的真实标签， $h_{\theta}(x)$ 是逻辑斯蒂回归的预测函数， θ 是模型的参数， α 是学习率。

加入L1正则化后，代价函数和梯度下降式可以修改为：

代价函数（加入L1正则化项）：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n |\theta_j|$$

$$\text{梯度下降式（加入L1正则化项）: } \theta_j := \theta_j - \alpha \left(\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \text{sign}(\theta_j) \right)$$

其中， λ 是正则化参数，控制正则化项的影响力。正则化项的目的是防止过拟合，通过对参数的绝对值进行惩罚来减小参数的值。

在梯度下降式中，除了原始的梯度项 $\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$ ，还加入了正则化项的梯度 $\frac{\lambda}{m} \text{sign}(\theta_j)$ 。正则化项的梯度根据参数的符号（正负）进行更新，使参数向零点靠近，并且在更新过程中保持参数的稀疏性。

加入L2正则化后，代价函数和梯度下降式可以修改为：

代价函数（加入L2正则化项）：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$$\text{梯度下降式（加入L2正则化项）: } \theta_j := \theta_j - \alpha \left(\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right)$$

其中， λ 是正则化参数，控制正则化项的影响力。正则化项的目的是防止过拟合，通过惩罚参数的平方和来减小参数的值。

在梯度下降式中，除了原始的梯度项 $\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$ ，还加入了正则化项的梯度 $\frac{\lambda}{m} \theta_j$ 。正则化项的梯度会使参数向较小的值进行更新，以减小参数的大小。

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Repeat until convergence

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)})$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

for $j = 1, 2, \dots, n$

}

四：决策树

决策树是一种用于分类和预测的机器学习模型。它由一系列的节点组成，其中每个内部节点测试一个属性，每个分支对应于属性的取值，每个叶节点指定一个分类。决策树可以通过逻辑规则的形式来重新表示，其中每个规则是对属性值实例的约束条件的合取范式。

决策树学习是指从训练数据中自动构建决策树模型的过程。在决策树学习中，我们根据已知的属性值和对应的分类标签来构建决策树模型。通过对属性值的比较和判断，决策树能够根据给定的输入样本进行分类或预测。

决策树的表达能力很强，它可以表示为一系列的测试条件，形式上是一个断言，即：

$\forall s \text{ WillWait}(s) \Leftrightarrow (P_1(s) \vee P_2(s) \vee \dots \vee P_n(s))$ 其中，每个条件 $P_i(s)$ 是一系列从根节点到正向叶节点的测试条件的合取范式。

决策树学习算法的目标是找到与训练集一致的决策树。对于每个训练实例，我们不能构建一棵决策树，其中每个路径都将属性值设置为实例的值，因为它只是记忆了训练实例，对于新实例无法进行泛化。

任何决策树算法的基本思想如下：

- 选择最佳的属性来划分剩余的实例，并将该属性作为决策节点。
- 递归地对每个子节点重复这个过程。
- 停止条件：
 - 所有实例具有相同的目标属性值。
 - 没有更多属性可用。
 - 没有更多实例可用。

熵

熵是一个度量集合无序程度的概念。在机器学习中，我们常常使用熵来度量数据的混乱程度。其公式为：

- 当我们只有正例 (p) 和负例 (n) 时，熵的计算公式为： $Entropy(D) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$ ，其中 D 代表数据集。
- 当我们有更多的类别时，熵的计算公式变为： $Entropy(D) = -\sum_{k=1}^y p_k \log_2 p_k$ ，其中 p_k 表示属于第 k 类的样本在总样本中的比例， y 是目标属性的范围的大小。

信息增益

信息增益是决策树选择分裂特征的重要依据，我们总是选择能最大化信息增益的特征进行分裂。信息增益是指通过某个特征划分数据集后，不确定性减少的程度。信息增益越高，意味着选择该特征可以获得更多的信息，使分类更准确。

信息增益是通过计算特征分裂前后熵的变化来定义的，计算公式为：

$Gain(D, a) = Entropy(D) - \sum_{v=1}^V \frac{|D_v|}{|D|} Entropy(D_v)$ ，其中 D 是数据集， a 是特征， D_v 是特征 a 取值为 v 的子集， V 是特征 a 的所有可能的取值。

决策树的目标是创建一棵对叶子节点的路径长度较短的树。理想情况下，一个完美的属性可以将样本分成全正或全负的子集，即最大的信息增益。信息增益的计算公式为：

$$Gain; D, a = Entropy(D) - \sum_{v=1}^V \frac{|D_v|}{|D|} Entropy(D_v)$$

在决策树的学习示例中，可以计算特征 'Type' 和 'Patrons' 的信息增益：

$$Gain; D, Type = 1 - \frac{2}{12} Entropy\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12} Entropy\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12} Entropy\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12} Entropy\left(\frac{1}{2}, \frac{1}{2}\right) = 0$$

$$Gain; D, Patrons = 1 - \frac{2}{12} Entropy(0, 1) + \frac{4}{12} Entropy(1, 0) + \frac{6}{12} Entropy\left(\frac{2}{6}, \frac{4}{6}\right) = 0.54$$

其中，"Patrons" 属性有最高的“信息增益”，因此选择它作为根节点。

ID3算法

ID3算法是一种用于生成决策树的贪心算法。算法从根节点开始，根据每个特征的信息增益来选择分裂特征。然后，对每个子节点递归应用这个过程，直到所有样本都被正确分类，或者没有更多的特征可以选择为止。

ID3算法具体步骤如下：

1. 从根节点开始，将所有数据都放在根节点上。
2. 对于每个节点，计算所有可能特征的信息增益。
3. 选择信息增益最高的特征。
4. 根据选定的特征将节点的数据进行划分（注意划分后的特征不能再被使用）。
5. 对每个叶节点递归地执行上述步骤，直到满足以下条件之一：
 - 所有的样本都被正确分类。
 - 没有可选的特征可用。

在每个节点上，信息增益用于选择最佳特征进行划分。

ID3算法通过递归地构建决策树，不断选择最佳特征进行划分，直到满足停止条件。这样，最终的决策树可以用于对新实例进行分类。

决策树算法（例如ID3）通常有一个偏好，那就是更喜欢短的树，并且喜欢让具有高信息增益的属性接近根节点。这种偏好就是“奥卡姆剃刀原则”，即在所有可能的解决方案中，最简单的解决方案最有可能是正确的。这也表达了对能够适应数据的最短假设的偏好。这种偏好是一种对某些假设（优先/搜索偏好）的偏好，而不是对假设空间的限制。

ID3存在一个问题，那就是越细小的分割分类错误率越小，所以ID3会**越分越细**，比如以第一个属性为例：设阈值小于70可将样本分为2组，但是分错了1个。如果设阈值小于70，再加上阈值等于95，那么分错率降到了0，但是这种分割显然只对训练数据有用，对于新的数据没有意义，这就是所说的过度学习（Overfitting）。

C4.5算法

C4.5算法是ID3的扩展。C4.5引入了信息增益率作为属性选择的标准，这考虑到了属性的固有价值（intrinsic value） IV_a ，其值越大，可能的属性值越多。信息增益率的公式为：

$$Gain_ratio(D, a) = \frac{Gain(D, a)}{IV(a)}$$

$$IV(a) = - \sum_{v=1}^V \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}$$

同样，最大信息增益率属性 a^* 被定义为：

$$a^* = \operatorname{argmax}_{a \in A} Gain_ratio(D, a)$$

C4.5中，优化项要除以分割太细的代价，这个比值叫做信息增益率，显然分割太细分母增加，信息增益率会降低。除此之外，其他的原理和ID3相同。

CART

进一步的，还有**CART**（Classification and Regression Tree）算法，这是由Breiman等人在1984年提出的。CART算法使用**基尼指数**（Gini Index）来衡量划分的质量。基尼指数 $Gini(D)$ 反映了从 D 中随机选取两个样本它们是不一致的的概率，其公式为：

$$Gini(D) = \sum_{k=1}^y \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^y p_k^2$$

GINI指数：即总体内包含的类别越杂乱，GINI指数就越大（跟熵的概念很相似）。

在CART算法中，对于某个属性 a ，其对应值为 v 的基尼指数定义为：

$$Gini_index(D, a) = v = \frac{|D_l|}{|D|} Gini(D_l) + \frac{|D_r|}{|D|} Gini(D_r)$$

其中， D_l 和 D_r 是根据属性 a 的值 v 分割 D 得到的子集。在CART算法中，最优属性和最优切分点 (a^*, v^*) 被定义为：

$$(a^*, v^*) = \operatorname{argmin}_{a \in A} Gini_index(D, a = v)$$

均方误差和CART算法

MSE and CART

$$a_*, v_* = \underset{a \in A}{\operatorname{argmin}} \left[\min_{c^l} \sum_{x^i \in D^l} (y^i - c^l)^2 + \min_{c^r} \sum_{x^i \in D^r} (y^i - c^r)^2 \right]$$
$$c_l = \frac{1}{N^l} \sum_{x^i \in D^l} y^i, \quad c_r = \frac{1}{N^r} \sum_{x^i \in D^r} y^i$$

D^l and D^r are the subsets of D splitted by $a = v$.

The CART can be applied to regression task!

总结如下:

算法	任务	准则	树结构	连续属性值	缺失属性值	剪枝	多重使用
ID3	分类	信息增益	多叉树	不支持	不支持	不支持	不支持
C4.5	分类	信息增益率	多叉树	支持	支持	支持	不支持
CART	分类; 回归	基尼指数; 均方误差	二叉树	支持	支持	支持	支持

ID3算法用于分类任务，使用信息增益作为划分准则，生成多叉树结构，不支持连续属性值和缺失属性值处理，也不支持剪枝和多重使用属性。

C4.5算法支持连续属性值和缺失属性值处理，并使用信息增益率作为划分准则，可以进行剪枝操作，但不支持多重使用属性。

CART算法既可以用于分类任务，也可以用于回归任务，使用基尼指数或均方误差作为划分准则，生成二叉树结构，同时支持连续属性值、缺失属性值处理、剪枝和多重使用属性。

树剪枝

树剪枝 (Pruning Trees) 是一种减少树中使用的属性数量的技术。通过移除子树可以获得更好的泛化性能，但这需要一个单独的剪枝集。树剪枝主要有两种类型:

1. 预剪枝 (Pre-pruning, 又叫前向剪枝)
2. 后剪枝 (Post-pruning, 又叫反向剪枝)

预剪枝 (Prepruning) 在构建过程中决定何时停止添加属性。例如，所有属性已经被使用；一个节点中的实例数少于某个阈值；精度不能提高。预剪枝可以降低过拟合的风险，但可能会产生欠拟合的问题。这是因为有时当前的一些分支的划分不能提高泛化性能，但基于它的后续划分可能导致显著的性能提升。

后剪枝 (Postpruning) 则等待完全的决策树构建完成后再进行剪枝。主要有三种方式：Reduced-Error pruning (REP)、Pesimistic-Error pruning (PEP)、Cost-Complexity pruning (CCP)。两种技术分别是：子树替换和子树提升。后剪枝的欠拟合风险较小，泛化性能通常优于预剪枝，但训练时间较长。

1. **Reduced-Error Pruning (REP)**：这是最简单的剪枝方法。首先，我们把数据集分成训练集和验证集。然后在训练集上生成完全的决策树，并用验证集来测试决策树的性能。在决策树的每个节点上，我们尝试将这个节点和其子树删除，然后看是否可以提高验证集上的性能。如果可以，我们就进行剪枝。
2. **Pessimistic-Error Pruning (PEP)**：这是一种自上而下的剪枝方法。对于决策树的每个节点，我们首先计算这个节点为叶节点时的误差（假设所有的子节点都被剪除），然后计算保持当前子树结构的误差。如果节点作为叶节点的误差更小，我们就剪除这个节点的所有子节点。
3. **Cost-Complexity Pruning (CCP)**：这是一种更复杂的剪枝策略，也被称为最小代价复杂度剪枝。它引入了一个复杂度参数，用于平衡决策树的复杂度（节点的数量）和训练数据上的误差。当增加一个新的叶节点不能显著降低训练误差时，我们就剪除这个节点。

子树替换 (Subtree Replacement) 是指将整个子树替换为单个叶节点，可以更多地泛化树，但可能会增加准确度。

子树提升 (Subtree Raising) 是指将整个子树提升到另一个节点。

特征缩放

特征缩放是指对特征数据进行线性变换，使其具有相似的尺度和范围。特征缩放可以提升某些机器学习算法的性能，特别是那些基于距离或梯度下降的算法。下面是特征缩放的一些要点和总结：

- 特征缩放的目的是消除特征之间的量纲和数量级差异，以确保它们在模型训练中的权重和影响具有相似的范围。
- 特征缩放可以提高模型的收敛速度和稳定性，并且能够更好地处理各特征之间的权衡和权重调整。
- 常见的特征缩放方法包括标准化（将特征缩放为均值为0，标准差为1的正态分布）、归一化（将特征缩放到[0,1]或[-1,1]的范围内）和对数转换等。

并不是所有算法都需要特征缩放：

- 对于基于梯度下降的算法，特征缩放可以使其更快地收敛，避免由于特征值差异过大而导致梯度更新过慢或不稳定的问题。
- 对于基于距离的算法，特征缩放可以确保不同特征对距离计算的贡献相当，避免由于特征尺度差异导致距离计算不准确或偏倚的问题。
- 对于基于树的算法，如决策树和随机森林，特征缩放通常不是必需的，因为它们并不关注特征的绝对值，而是基于特征的排序和划分。
- 一些算法（如线性判别分析和朴素贝叶斯）在其模型中已经内置了特征缩放的机制，因此不需要显式进行特征缩放。

思考题

决策树算法是否要做特征缩放？

在一般情况下，决策树算法不需要进行特征缩放。决策树算法的原理是通过选择最佳的属性进行划分，而特征的缩放不会影响属性的划分。因此，对于决策树算法来说，特征的绝对值或尺度并不重要。

相比于其他机器学习算法（如支持向量机或神经网络），决策树算法对特征缩放不敏感。决策树是基于属性值的排序和划分，而不是依赖于特征的具体数值。

决策树总是横向的分割，即使某一个变量成比例的扩大倍数也没有关系。线性回归之所以不受影响的是因为线性方程的比例关系总是不变的，所以缩放对其没有影响。

五：神经网络

大脑由大量相互连接的神经元构成（大约有 10^{11} 个神经元，每个神经元平均有 10^4 个连接）。每个神经元与许多其他神经元连接。

神经元通过其突触从其他神经元接收输入，输入大致相加。当输入超过某个阈值时，神经元发送一个电信号，该信号从细胞体沿着轴突传播到下一个神经元。

人工神经元

人工神经元（Artificial Neuron）的设计受到了生物神经元的启发。其公式为： $y = \varphi(\sum w_i x_i - b)$ ，其中 w_i 代表权重， x_i 是输入， b 是偏置， φ 是激活函数。

Rosenblatt的单层感知器是早期的神经网络模型，它的更新规则可以写成：

$$b = b + \eta(y - \hat{y})$$

$$w_i = w_i + \eta(y - \hat{y})x_i$$

其中， y 是实际输出， $\hat{y}$ 是预测输出， η 是学习率。如果输出正确，则不进行任何操作。如果输出过高，降低活动输入的权重。如果输出过低，增加活动输入的权重。

由于感知器使用线性阈值函数，它在寻找可以区分类别的线性分隔符。如果训练数据可以线性分隔，并且学习率 η 足够小，感知器的训练可以收敛。然而，**单层感知器只能学习线性可分的概念**。

当感知器加入隐藏层时，就能够表示所有的布尔函数，即两层网络可以表示任意复杂度的布尔函数。

激活函数

为了超越线性函数，我们需要非线性的输出函数。虽然可以创建多层的线性网络，但这样的网络仍然是线性的。因此，我们需要一个连续的、可微的、非线性的并且易于计算的激活函数。

一个常用的激活函数是逻辑斯特函数（Logistic Sigmoid），其函数和导数分别为：

$$\sigma z = \frac{1}{1+e^{-z}} \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

其输出范围是 $[0,1]$ ，由生物神经元激发并且可以被解释为给定其输入的人工神经元'firing'的概率。然而，饱和的神经元会导致梯度消失。

有了非线性的激活函数，感知器就可以表示更加复杂的模型。

给定一个由（数据，标签）对组成的训练数据集，我们让机器学习从数据到标签的函数。函数集合被称为假设空间，学习过程被称为更新参数使得预测值接近相应的标签。

$$D = x_i, y_i \quad i = 1, 2, \dots, N \quad y^{(i)} \approx f_{\theta} x_i$$

其中， θ 代表神经网络中的所有权重和偏置。不同的连接方式会导致不同的网络结构。

全连接前馈神经网络（Fully Connected Feedforward Network）是一种常见的神经网络结构，由输入层、若干个隐藏层和输出层组成。在给定网络结构的情况下，我们可以定义一个函数集合，不同的权重和偏置会导致不同的函数。

神经网络训练步骤

1. 定义学习目标

学习目标就是通过定义损失函数来度量预测值与实际标签值之间的误差。对于回归问题，最常见的损失函数是平方损失函数，形式如下：

$$L(y(i), f(x_i; \theta)) = \frac{1}{2} (y(i) - f(x_i; \theta))^2$$

其中， $y(i)$ 是样本的实际标签， $f(x_i; \theta)$ 是模型的预测结果， θ 表示模型参数。

参数优化

为了优化参数，我们需要使用优化算法，如梯度下降法、共轭梯度法、拟牛顿法等。其中，梯度下降法是最常见的一种方法。在每次迭代中，我们沿着损失函数的负梯度方向更新参数，以使损失函数的值逐渐减小。具体地，参数的更新公式如下：

$$\theta_i := \theta_i - \alpha \frac{\partial J(\theta)}{\partial \theta_i}$$

其中， α 是学习率， $\frac{\partial J(\theta)}{\partial \theta_i}$ 是损失函数关于参数 θ_i 的梯度。

反向传播算法

在神经网络中，为了计算梯度，我们使用了反向传播算法。该算法包含正向传递和反向传递两个步骤：

- 正向传递：在正向传递阶段，我们按照从输入层到输出层的顺序，计算并存储每个神经元的输入和输出值。
- 反向传递：在反向传递阶段，我们从输出层开始，向输入层方向，计算每个神经元输出值对损失函数的梯度，并将这些梯度传递给它的输入神经元。

具体来说，对于激活函数的输入 z ，其梯度可以通过链式法则来计算：

$$\frac{\partial C}{\partial z} = \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z}$$

其中， C 是损失函数， a 是激活函数的输出， $\frac{\partial C}{\partial a}$ 是激活函数的输出对损失函数的梯度， $\frac{\partial a}{\partial z}$ 是激活函数的导数。

损失函数关于连接权重 w 的梯度(偏导数)可以通过链式法则计算得到：

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial z} \frac{\partial z}{\partial w} = \frac{\partial C}{\partial a} \sigma'(z) * a = \frac{\partial C}{\partial a} * a * \sigma'(z)。$$

以一个神经元为例，假设其输入为 z ，权重为 w ，激活函数为 σ ，输出为 $a = \sigma(z)$ ，损失函数为 C ，则损失函数关于 z 的偏导数为 $\frac{\partial C}{\partial z} = \frac{\partial C}{\partial a} \sigma'(z)$ 。

然后，损失函数关于权重 w 的偏导数可以通过链式法则计算得到：

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial z} \frac{\partial z}{\partial w} = \frac{\partial C}{\partial a} \sigma'(z) * a = \frac{\partial C}{\partial a} * a * \sigma'(z)。$$

- 训练复杂度：人工神经网络需要进行大量的训练，并且有许多参数需要调整。
- 收敛性：反向传播算法只能保证网络收敛到某个误差的局部最小值，而不能保证达到全局最小值。
- 强大性能：反向传播算法非常强大，只要有足够的隐藏单元，它能够学习任何函数。

BP神经网络

BP神经网络的基本构造包括输入层、隐含层和输出层三个部分。输入层负责接收输入信息并传递给隐含层，隐含层进行信息的处理和计算，输出层将处理后的信息输出。这三个部分都包含若干个神经元，并通过权重连接在一起。

BP神经网络的训练包括前向传播和反向传播两个过程：

1. **前向传播**：输入样本从输入层进入，通过隐含层，并最终得到输出层的输出。这个过程是依据各层神经元的连接权重进行计算和传递。
2. **反向传播**：根据输出层的实际输出与期望输出（即训练样本的标签）的差距，计算出误差，然后将这个误差反向传递到隐含层，再根据误差调整各层神经元间的连接权重。

这个过程反复进行，直到网络的输出误差达到预设的容忍范围，或者达到预设的最大训练次数。的基本构造包括输入层、隐含层和输出层三个部分。输入层负责接收输入信息并传递给隐含层，隐含层进行信息的处理和计算，输出层将处理后的信息输出。这三个部分都包含若干个神经元，并通过权重连接在一起。

BP神经网络的训练包括前向传播和反向传播两个过程：

1. **前向传播**：输入样本从输入层进入，通过隐含层，并最终得到输出层的输出。这个过程是依据各层神经元的连接权重进行计算和传递。
2. **反向传播**：根据输出层的实际输出与期望输出（即训练样本的标签）的差距，计算出误差，然后将这个误差反向传递到隐含层，再根据误差调整各层神经元间的连接权重。

这个过程反复进行，直到网络的输出误差达到预设的容忍范围，或者达到预设的最大训练次数。

人工神经网络 (Artificial Neural Network, ANN) 的关键要点:

- 1. 神经网络的表征能力:** 神经网络具有强大的表征能力, 可以实现任意函数的近似。
 - 布尔函数: 可以通过两层网络 (足够的隐藏单元) 来表示任意布尔函数。
 - 连续函数: 任意有界连续函数可以通过两层网络 (足够的隐藏单元) 进行近似。
 - 任意函数: 任意函数可以通过三层网络进行近似。
- 2. 神经网络的标准结构:** 神经网络主要由输入单元、隐藏单元和输出单元组成, 这些单元之间通过链接相连, 每个链接都有一个数字权重。
 - 输入单元: 以固定长度的数字向量表示输入。
 - 隐藏单元: 代表网络学习的中间计算过程。通常只有一层 (即两层网络)。
 - 输出单元: 以固定长度的数字向量表示输出。
- 3. 神经网络的建立:** 在构建神经网络时, 需要决定以下几点:
 - 单元的数量 (超参数): 在隐藏层中使用多少个隐藏单元? 太少则无法学习, 太多则泛化能力差。
 - 需要的单元类型 (超参数)。
 - 单元之间的连接方式。
- 4. 确定隐藏层节点个数:** 隐藏单元过少会导致网络无法适应数据, 过多则可能导致过拟合。可以通过内部交叉验证来确定隐藏单元的最优数量。
- 5. 神经网络的学习:** 学习过程包括初始化连接权重, 使用学习算法, 并更新连接权重以适应训练数据集。
- 6. 前馈神经网络:** 前馈神经网络 (Feedforward Neural Network) 是一种基本的神经网络模型, 由输入层、隐藏层 (可选) 和输出层组成。信息在网络中沿着前向的方向传播, 从输入层经过隐藏层到输出层, 不形成环路。每个神经元接收上一层的输出作为输入, 并应用激活函数计算输出。前馈神经网络的特点是信息只向前流动, 没有反馈回路。

一个典型的**多层前馈神经网络**包括输入层、隐藏层和输出层, 每层都与下一层全连接, 激活函数向前传播。
- 7. 误差反向传播:** 首先计算输出单元的误差, 并用此改变顶层的权重, 然后基于输出单元的误差来计算隐藏单元的误差, 最后根据隐藏单元的误差更新底层的权重。
 - 输出层误差计算公式: $\delta_j = o_j(1 - o_j)(t_j - o_j)$
 - 隐藏层误差计算公式: $\delta_j = o_j(1 - o_j) \sum_k \delta_k w_{jk}$
 - 权重更新公式: $\Delta w_{ij} = \eta \delta_j o_i$

其中, o_j 是当前输出, t_j 是期望输出, δ_j 是 j 单元的误差, η 是学习率, o_i 是 i 单元的输出, w_{ij} 是 i 单元到 j 单元的连接权重。
- 8. 反向传播算法:** 反向传播是一种有效的神经网络训练方法, 它通过计算网络的输出错误, 并将这些错误反向传播到网络的每一层, 从而更新每层的权重。

算法的步骤如下:

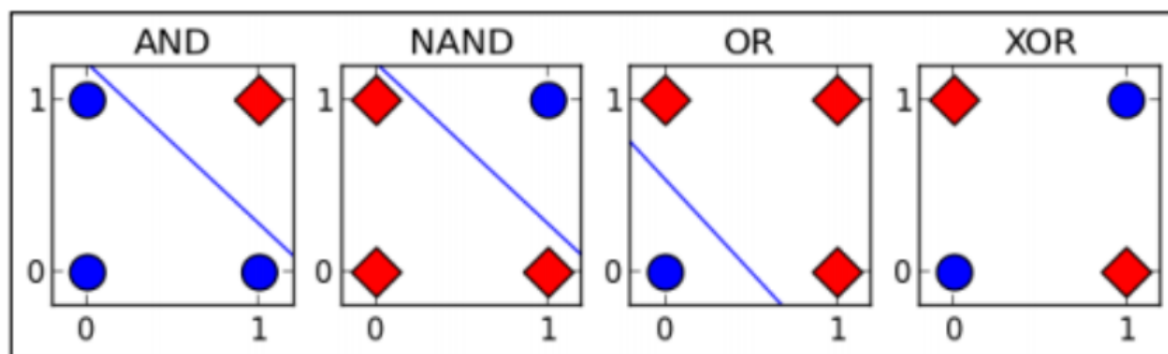
 - 创建一个多层网络, 其中隐藏层有 H 个隐藏单元, 每一层之间都全连接, 权重被设置为小的随机实数。
 - 训练过程持续进行, 直到所有训练样本的输出值达到正确值 (误差在 ϵ 以内), 或完成了足够多的训练轮次, 或者均方误差停止下降, 或满足其他终止条件。
 - 每一个训练轮次开始时, 对每一个训练样本, 都计算网络对应于该样本输入的输出, 计算当前输出和正确输出之间的误差, 并通过反向传播误差以及应用学习规则来更新权重。使用**链式法则**来计算每一层的梯度, 并根据梯度的方向调整权重。

请注意, 这是随机梯度下降版本的算法, 它在每个训练样本上更新权重。
- 9. 反向传播算法的评价:**
 - 反向传播算法非常强大, 给定足够的隐藏单元, 它可以学习任何函数。
 - 确定隐藏单元的数量是困难的。
 - 神经网络需要大量的训练, 有许多参数需要调整。
 - 训练可能会非常慢。
 - 确定何时终止权重更新循环的合适条件是一个问题。
- 10. 隐藏单元的代表:** 训练后的隐藏单元可以被看作是新构建的特征, 它们使得目标概念在转换的空间中线性可分。在许多实际领域中, 隐藏单元可以被解释为有意义的特征, 如元音检测器或边缘检测器等。然而, 隐藏层也可以成为输入的分布式表示, 在这种表示中, 每个单独的单元不容易被解释为一个有意义的特征。
- 11. 神经网络的主要概念:**

- 神经网络是一个计算模型，模拟人脑的一些属性。
- 单元的连接和性质决定了神经网络的行为。
- 感知机是一种前馈网络，只能表示线性可分的函数。
- 给定足够的单元，任何函数都可以由多层前馈网络表示。
- 反向传播学习算法适用于多层前馈网络。
- 神经网络被广泛应用于开发人工学习系统。

思考题

为什么只有输入输出层的神经网络解决不了异或问题？后来是如何解决的？



单层神经网络（只有输入和输出层）无法解决异或问题是因为异或函数不是线性可分的。线性可分意味着可以通过一条直线或一个超平面将不同类别的数据分开，而异或函数的数据点无法用单个直线或超平面进行分割。

为了解决这个问题，人们引入了多层神经网络，特别是具有**隐藏层**的多层前馈神经网络。隐藏层的存在使神经网络能够学习更复杂的非线性关系，并且能够表示非线性可分的函数。

通过使用隐藏层，多层神经网络可以通过组合多个简单的线性函数和非线性激活函数来近似任意复杂度的函数。通过反向传播算法，可以调整网络中的权重和偏置，使网络能够逐渐学习到异或函数的映射关系。

六：深度学习

与传统机器学习的区别

深度学习和传统的机器学习在以下几个方面有所不同：

1. **数据表示**：传统机器学习方法通常依赖于手工设计的特征表示，即将原始数据转换为具有区分性的特征向量。而深度学习通过神经网络自动从原始数据中学习表示，无需手动提取特征。
2. **模型复杂性**：深度学习模型通常由多个层级的神经元组成，形成深层结构，因此可以学习更复杂的函数关系。传统机器学习方法通常使用简单的模型，如线性回归、决策树等。
在复杂任务中，即使是非常相似的输入，也可能产生不同的输出；反之，非常不同的输入，也可能产生相似的输出。因此，我们需要进行**更高层次和更抽象层次的学习**。
3. **特征工程**：传统机器学习方法中，特征工程是一个重要的步骤，需要手动选择、构造和优化特征。与传统的手动特征工程不同，深度学习可以通过端到端的训练，从数据中自动学习特征，这就消除了手动设计和选择特征的需要，减少了对特征工程的依赖。
4. **端到端学习**：在深度学习中，我们通常采用一种称为端到端学习的方法。这种方法直接从输入数据学习到输出，所有的函数都是从数据中学习得来的。例如，在图像识别中，所有的特征（例如f1, f2, f3, f4等）都是从数据中学习的。这种方法减少了工程劳动力，但机器学习的内容更多。
5. **训练方式**：传统机器学习方法使用基于优化算法的迭代训练，如梯度下降等。而深度学习通常使用反向传播算法进行训练，该算法有效地计算了模型参数对于损失函数的梯度，从而实现了高效的模型训练。
6. **数据需求**：深度学习通常需要大量的标注数据进行训练，因为深度神经网络参数众多，需要足够的数据来调整这些参数。传统机器学习方法对于数据量的要求相对较低。
7. **可解释性**：传统机器学习方法通常具有较好的可解释性，可以通过特征权重等方式解释模型的决策依据。而深度学习模型由于其复杂性，往往难以解释其内部的工作原理和决策依据。

8. **联合学习与分离学习**：在传统的机器学习过程中，通常会有多个步骤，包括数据收集、预处理、特征提取、分类器的训练等，这些步骤通常是分开进行的。但在深度学习中，我们可以同时进行特征变换和分类，这是深度学习的力量所在。它通过联合学习多个非线性变换（即特征）和分类器，从而实现更好的性能。

总的来说，深度学习是一种强大的机器学习方法，它能够自动地从数据中学习特征，并进行端到端的学习，这大大减少了手动设计和选择特征的需要，同时提高了学习效果。但同时深度学习也对数据量和计算资源的需求较高。传统机器学习方法在一些小数据集和可解释性要求较高的场景下仍然具有优势。

深度学习而不是宽度学习

- 提升同样效果需要增加的宽度远远超过需要增加的深度。在神经元数目相同的情况下，“深度”学习出现的情况会更多，效果也会更好、更全面。
 - 宽而浅的网络可能比较**擅长记忆**，却不擅长概括，即**泛化能力差**。而网络的深度带来的是逐层抽象的能力。
 - 深度和宽度并不是完全对立，二者都是在增加可学习参数的个数，从而增加神经网络的拟合能力，在网络设计时，两者都会考虑，追求深度与广度的平衡。
- 胖+短 vs 瘦+高**：对于具有相同参数数量的浅层网络（胖+短）和深层网络（瘦+高），深层网络往往能够提供更好的性能。理论上，一个浅层的宽大网络能够学习到一个深层瘦小网络能够学习到的所有函数，但在实践中，深度网络通常能够以更少的参数数量更好地泛化。
 - 深度优于宽度**：根据一些研究，比如Seide等人在2011年的"Conversational Speech Transcription Using Context-Dependent Deep Neural Networks"，他们发现深度网络往往表现得更好。比如，他们发现4层的神经网络比只有1层的神经网络表现得更好，尽管神经元总数量相同。
 - 模块化**：深度网络能够更好地利用模块化。模块化意味着网络可以在各个层级学习并表示不同级别的抽象，从而提高学习效率和表现力。例如，对于图像分类任务，第一层可以学习到简单的边缘和纹理特征，第二层可以组合这些特征学习到更复杂的形状特征，以此类推。
 - 深度学习的灵感来源**：深度学习的设计灵感主要来源于生物视觉系统，特别是哺乳动物的视觉皮层。视觉皮层由多层神经元组成，每层神经元在处理视觉信息时都会进行某种形式的转换或抽象。这种层次化的处理方式被深度学习采纳并用于处理各种类型的数据。
 - 深度等于隐藏层的多样性**：从实际应用来看，更深的神经网络（即具有更多隐藏层的神经网络）通常可以取得更好的性能。例如，AlexNet有8层，VGG有19层，GoogleNet有22层，而最新的Residual Net甚至有152层，随着层数的增加，错误率也在持续下降。

总的来说，“深度学习”之所以被称为“深度”，主要是因为它的多层结构可以更好地表示和学习数据的复杂性和层次性。

如何进行深度学习？

机器学习框架

在机器学习中，我们通常通过最小化训练集上的预测错误来估计预测函数 f ，并将 f 应用于从未见过的测试样例，以获得预测结果。

深度学习的三个步骤

- 定义函数集**：在深度学习中，我们通过定义网络结构来定义一组函数。例如，对于全连接前馈网络，我们可以通过不同的权重 w 和偏置 b 来定义不同的函数。
- 度量函数的好坏**：我们需要一个方法来度量函数的好坏。在深度学习中，我们通常使用损失函数（如交叉熵）来度量预测与目标之间的差距。
- 选择最好的函数**：我们需要一种方法来在函数集中找到最好的函数。在深度学习中，我们通常使用梯度下降方法来最小化损失函数，从而找到使得预测误差最小的函数。

神经网络

在深度学习中，我们常常使用神经网络作为我们的函数集。一个神经网络可以由输入层、隐藏层和输出层组成。输入层负责接收输入特征，隐藏层负责从输入特征中抽取高级特征，而输出层负责基于这些特征进行预测。

损失函数

在深度学习中，我们常常使用损失函数来度量预测与目标之间的差距。常见的损失函数有均方误差、交叉熵等。

梯度下降

这便是深度学习中的“学”。在深度学习中，我们常常使用梯度下降方法来最小化损失函数。在梯度下降中，我们首先计算损失函数关于每个参数的梯度，然后用这个梯度来更新参数。

反向传播

在神经网络中，我们通常使用反向传播算法来高效地计算损失函数关于权重的梯度。这个算法利用了链式法则，从输出层开始，逐层向前计算梯度。

深度学习训练的挑战：

1. **训练集表现好但测试集表现差**：这可能是由于过拟合（overfitting）引起的。过拟合是指模型在训练数据上的表现很好，但在未见过的测试数据上的表现却很差。这通常是因为模型过于复杂，以至于它记住了训练数据中的噪声，而没有学习到真正的规律。

对于过拟合的问题，可以尝试使用正则化（regularization），dropout，早停（early stopping）等方法来减轻。

2. **网络深度增加不一定带来更好的效果**：理论上，增加网络的深度可以提高模型的表示能力，但在实践中，这可能会带来一些问题。例如，训练深层网络时可能会遇到梯度消失（vanishing gradient）或梯度爆炸（exploding gradient）的问题。

对于梯度消失问题，可以尝试使用如ReLU，Leaky ReLU，Parametric ReLU和Maxout等新的激活函数来缓解。

3. **梯度消失问题**：在深度神经网络中，反向传播过程中的梯度可能会随着层数的增加而快速衰减，导致较低层的参数更新变得非常缓慢。这就是梯度消失问题。

解决梯度消失问题的方法包括使用ReLU及其变体作为激活函数，使用批归一化（Batch Normalization）等。

Dropout

Dropout是一种在深度学习中使用的正则化技术，其通过在训练过程中随机丢弃一部分神经元的输出来减少过拟合。下面是有关Dropout的总结：

1. 训练过程中的Dropout：

- Dropout被应用在训练过程中，其中每个神经元有一定的概率被丢弃（设置为0），概率由dropout率 p 确定。
- 对于每个训练样本，随机选择一些神经元进行丢弃，这样在每次迭代中都会有不同的神经元被丢弃，模型变得更加鲁棒。
- 丢弃的神经元不参与前向传播和反向传播过程，因此模型在每次迭代中相当于是随机的子网络上训练。
- 为了在训练过程中保持期望的输出，每个保留的神经元的输出需要除以 $1-p$ 进行缩放。

2. 测试过程中的Dropout：

- 在测试过程中，所有的神经元都保留，不进行丢弃操作。
- 为了保持训练时期望输出的一致性，每个神经元的输出权重需要乘以 $(1-p)$ 。
- 这样可以确保在测试过程中，所有的神经元都参与了前向传播，获得了模型在整个训练过程中的平均效果。

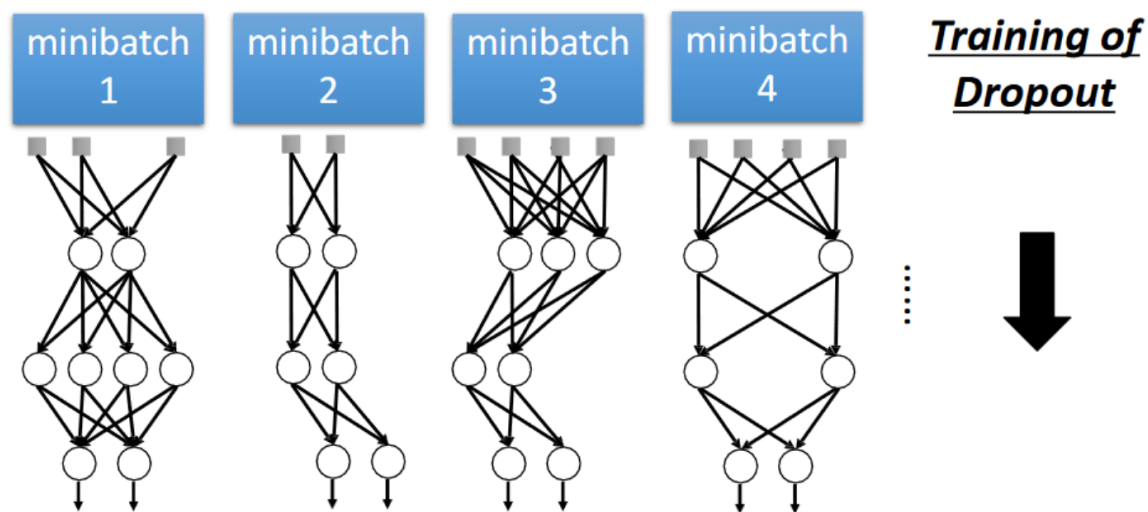
3. Dropout的直观原理：

- Dropout的直观原理类似于合作学习的情景，如果每个人都期望合作伙伴会完成工作，最终什么也不会做。
- 但是，如果你知道你的合作伙伴可能会退出，你会做得更好。
- 在测试过程中，实际上没有任何一个神经元被丢弃，所以最终能够获得好的结果。

4. Dropout与集成网络：

- Dropout还可以看作是一种集成网络的方法。
- 训练过程中，通过丢弃一部分神经元，每次迭代都相当于在训练不同的子网络。
- 最终的模型可以看作是许多不同结构的网络的集成，这有助于提高模型的泛化能力和鲁棒性。

Dropout is a kind of ensemble.



梯度消失/爆炸

目前优化神经网络的方法都是基于BP，即根据损失函数计算的误差通过梯度反向传播的方式，指导深度网络权值的更新优化。其中将误差从未层往前传递的过程需要**链式法则 (Chain Rule)** 的帮助，因此反向传播算法可以说是梯度下降在链式法则中的应用。

而链式法则是一个**连乘的形式**，所以当层数越深的时候，梯度将以指数形式传播。梯度消失问题和梯度爆炸问题一般随着网络层数的增加会变得越来越明显。在根据损失函数计算的误差通过梯度**反向传播**的方式对深度网络权值进行更新时，得到的**梯度值接近0或特别大**，也就是**梯度消失或爆炸**。梯度消失或梯度爆炸在本质原理上其实是一样的。

产生梯度消失和梯度爆炸的原因

【**梯度消失**】经常出现，产生的原因有：一是在**深层网络**中，二是采用了**不合适的损失函数**，比如sigmoid。当梯度消失发生时，接近于输出层的隐藏层由于其梯度相对正常，所以权值更新时也就相对正常，但是当越靠近输入层时，由于梯度消失现象，会导致靠近输入层的隐藏层权值更新缓慢或者更新停滞。这就导致在训练时，只等价于后面几层的浅层网络的学习。

【**梯度爆炸**】一般出现在**深层网络**和**权值初始化值太大**的情况下。在深层神经网络或循环神经网络中，**误差的梯度可在更新中累积相乘**。如果网络层之间的**梯度值大于 1.0**，那么**重复相乘会导致梯度呈指数级增长**，梯度变的非常大，然后导致网络权重的大幅更新，并因此使网络变得不稳定。

梯度爆炸会伴随一些细微的信号，如：

- ①模型不稳定，导致更新过程中的损失出现显著变化；
- ②训练过程中，在极端情况下，权重的值变得非常大，以至于溢出，导致模型损失变成 NaN 等等。

思考题

什么是梯度消失？

梯度消失是指在深度神经网络中，当反向传播算法计算梯度时，梯度值逐渐变小并趋近于零的现象。这导致在更新网络参数时，较浅层的神经元接收到的梯度信息非常小，无法有效更新参数，从而使得网络学习能力受限。

梯度消失的主要原因是深层网络中使用了一些激活函数（如Sigmoid函数），这些函数在输入很大或很小的情况下，导数接近于零。当反向传播算法向浅层传播时，梯度通过多个层次相乘，导致梯度逐渐变小。

梯度消失有哪些解决方法？

1. 激活函数选择：使用非线性的激活函数可以缓解梯度消失问题。传统的Sigmoid和Tanh函数在输入较大或较小的区域梯度接近于零，可以考虑使用ReLU、Leaky ReLU、ELU等激活函数，它们在一定范围内具有较大的梯度，能够更好地传递梯度信号。
2. 参数初始化：合适的参数初始化可以帮助减轻梯度消失问题。如果权重初始化过大，激活值可能会趋向于极端值，导致梯度消失。通常可以采用一些特定的初始化方法，如Xavier初始化、He初始化等，使得权重初始化在合理范围内。
3. 批归一化（Batch Normalization）：批归一化通过在每一层的输入上进行归一化操作，使得每个神经元的输入具有相对较小的变化范围，有助于缓解梯度消失问题。
4. 残差连接（Residual Connections）：在深层网络中引入残差连接可以将浅层的梯度直接传递到深层，避免了梯度消失。这种方法被广泛应用于ResNet等深度残差网络结构中。
5. 长短期记忆网络（LSTM）和门控循环单元（GRU）：这些递归神经网络结构通过门控机制有效地捕捉和传播长期依赖关系，从而缓解了梯度消失问题。

七：SVM

在二维空间上，两类点被一条直线完全分开叫做**线性可分**。

从二维扩展到多维空间中时，将 D_0 和 D_1 完全正确地划分开的 $wx + b = 0$ 就成了一个超平面。

为了使这个超平面更具鲁棒性，我们会去找最佳超平面，以最大间隔把两类样本分开的超平面，也称之为最大间隔超平面。

- 两类样本分别分割在该超平面的两侧；
- 两侧距离超平面最近的样本点到超平面的距离被最大化了。

支持向量机（SVM）是一种二分类模型，其基本模型定义为特征空间上间隔最大的线性分类器。支持向量机的学习策略是最大化间隔，并将其转化为一个凸二次规划问题的求解。

SVM 想要的就是找到各类样本点到超平面的距离最远，也就是找到最大间隔超平面。使得两类样本在该超平面两侧，且各类样本到超平面的最短距离（即函数间隔）最大。如果存在多个这样的超平面，SVM选择函数间隔最大的那个。

超平面

超平面 (Hyperplane)：在 n 维空间中，一个超平面是一个 $n - 1$ 维的子空间，其可以用以下公式定义：

$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0$ ，其中 $\mathbf{w}$ 是法向量， b 是偏置项。

假设超平面可以表示为： $f(x) = w^T x + b$ 。其中， w 是超平面的法向量， b 是截距。为了求解这个问题，我们可以使用拉格朗日乘子法，并引入KKT条件，得到一个对偶问题。

在对偶问题中，我们优化的是关于拉格朗日乘子的函数，即：

$$\min_{\alpha, \beta} \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \alpha_i [y_i (w^T x_i + b) - 1]$$

求解得到 α_i 后，可以得到 w 和 b 的解，从而得到分隔超平面。

硬间隔最大化 (Hard Margin Maximization)：对于线性可分数据集，SVM通过最大化间隔（即支持向量到超平面的距离）来找到最佳的划分超平面，数学表达如下： $\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$ s.t. $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, i = 1, \dots, m$

支持向量 (Support Vector)：最终确定超平面的样本点称为支持向量，它们满足： $y_i (\mathbf{w}^T \mathbf{x}_i + b) = 1$

软间隔支持向量机

在现实问题中，我们常常遇到线性不可分的情况，此时可以使用**软间隔**支持向量机。对于软间隔，我们**允许一些样本点不满足约束条件**，即允许一些样本点位于分隔超平面的错误一侧。为此，我们引入**松弛变量** ξ_i ，并在目标函数中加入这些松弛变量的和。这样，原问题变为：

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

其中， $C > 0$ 是一个参数，用于控制最大化间隔和最小化训练错误的权衡。

拉格朗日对偶问题 (Lagrangian Dual Problem): 将上述优化问题转化为其对偶形式, 求解对偶问题得到的解可以得到原问题的解, 公式如下:

$$\max_{\alpha} \min_{\mathbf{w}, b, \xi} L(\mathbf{w}, b, \xi, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$$

由此我们将不等式约束转化为了等式约束, 并得到 Lagrange 函数:

$$\begin{aligned} L(w, \lambda, a) &= f(w) + \sum_{i=1}^n \lambda_i h_i(w) \\ &= f(w) + \sum_{i=1}^n \lambda_i [g_i(w) + a_i^2] \quad \lambda_i \geq 0 \end{aligned}$$

由等式约束优化问题极值的必要条件对其求解, 联立方程:

$$\begin{cases} \frac{\partial L}{\partial w_i} = \frac{\partial f}{\partial w_i} + \sum_{i=1}^n \lambda_i \frac{\partial g_i}{\partial w_i} = 0, \\ \frac{\partial L}{\partial a_i} = 2\lambda_i a_i = 0, \\ \frac{\partial L}{\partial \lambda_i} = g_i(w) + a_i^2 = 0, \\ \lambda_i \geq 0 \end{cases}$$

(为什么取 $\lambda_i \geq 0$, 可以通过几何性质来解释, 有兴趣的同学可以查下 KKT 的证明)。

针对 $\lambda_i a_i = 0$ 我们有两种情况:

情形一: $\lambda_i = 0, a_i \neq 0$

由于 $\lambda_i = 0$, 因此约束条件 $g_i(w)$ 不起作用, 且 $g_i(w) < 0$

情形二: $\lambda_i \neq 0, a_i = 0$

此时 $g_i(w) = 0$ 且 $\lambda_i > 0$, 可以理解为约束条件 $g_i(w)$ 起作用了, 且 $g_i(w) = 0$

综合可得: $\lambda_i g_i(w) = 0$, 且在约束条件起作用时 $\lambda_i > 0, g_i(w) = 0$; 约束不起作用时 $\lambda_i = 0, g_i(w) < 0$

由此方程组转换为:

$$\begin{cases} \frac{\partial L}{\partial w_i} = \frac{\partial f}{\partial w_i} + \sum_{j=1}^n \lambda_j \frac{\partial g_j}{\partial w_i} = 0, \\ \lambda_i g_i(w) = 0, \\ g_i(w) \leq 0 \\ \lambda_i \geq 0 \end{cases}$$

以上便是不等式约束优化问题的 **KKT(Karush-Kuhn-Tucker)** 条件, λ_i 称为 KKT 乘子。

KKT条件是确保鞍点是原函数最优解的充分条件。

遇到低维完全不可分样本时，可以将二维线性不可分样本映射到高维空间中，让样本点在高维空间线性可分。低维空间映射到高维空间后维度可能会很大，如果将全部样本的点乘全部计算好，这样的计算量过大。但如果我们有这样的一核函数， x_i 和 x_j 在特征空间的内积等于它们在原始样本空间中通过函数 $k(x,y)$ 计算的结果就不需要计算高维甚至无穷维空间的内积了。

核函数

核函数 (Kernel Function): 对于非线性问题，我们通过核函数将输入空间映射到一个高维空间，使得样本在新的空间中变得线性可分。常用的核函数有：线性核、多项式核、高斯核等。

- 线性核函数 (Linear Kernel) :** $K(x, y) = x^T y$ 线性核函数在特征空间中进行内积计算，相当于没有进行映射，直接在原始特征空间中进行分类。
适用于**线性可分**的问题，即数据在原始特征空间中可以被一个超平面完全分开。
- 多项式核函数 (Polynomial Kernel) :** $K(x, y) = (x^T y + c)^d$ 多项式核函数将数据映射到更高维的特征空间，其中 c 是常数项， d 是多项式的次数。
适用于数据在特征空间中具有**一定的非线性关系**的问题，多项式核函数的次数 d 可以调节映射的复杂度。多项式核函数在原始特征空间中进行多项式映射计算，计算效率相较于高斯核函数较高。
- 高斯核函数 (Gaussian Kernel 或 RBF核函数) :** $K(x, y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$ 高斯核函数通过计算数据点之间的欧氏距离，将数据映射到无穷维的特征空间。 σ 是高斯核函数的带宽参数，控制了数据点在特征空间中的相似度。
适用于数据在特征空间中具有**复杂非线性关系**的问题。高斯核函数的带宽参数 σ 控制了相似度的衰减速度，需要根据数据分布进行调节。
- Sigmoid核函数:** $K(x, y) = \tanh(\alpha x^T y + c)$ Sigmoid核函数通过计算数据点之间的相似度，将数据映射到一个非线性的特征空间。 α 和 c 是调节参数。
适用于某些特定的非线性问题，但相对于其他核函数，Sigmoid核函数的性能较差

SMO算法

SMO算法 (Sequential Minimal Optimization): 是一种常用的优化算法，用于求解支持向量机 (SVM) 的对偶问题。该算法的核心思想是每次选择一对拉格朗日乘子 (α_i 和 α_j)，通过解析方式优化它们，以逐步迭代地最小化目标函数。这样的选择和优化过程重复进行，直到达到收敛条件。

SMO算法的具体步骤如下：

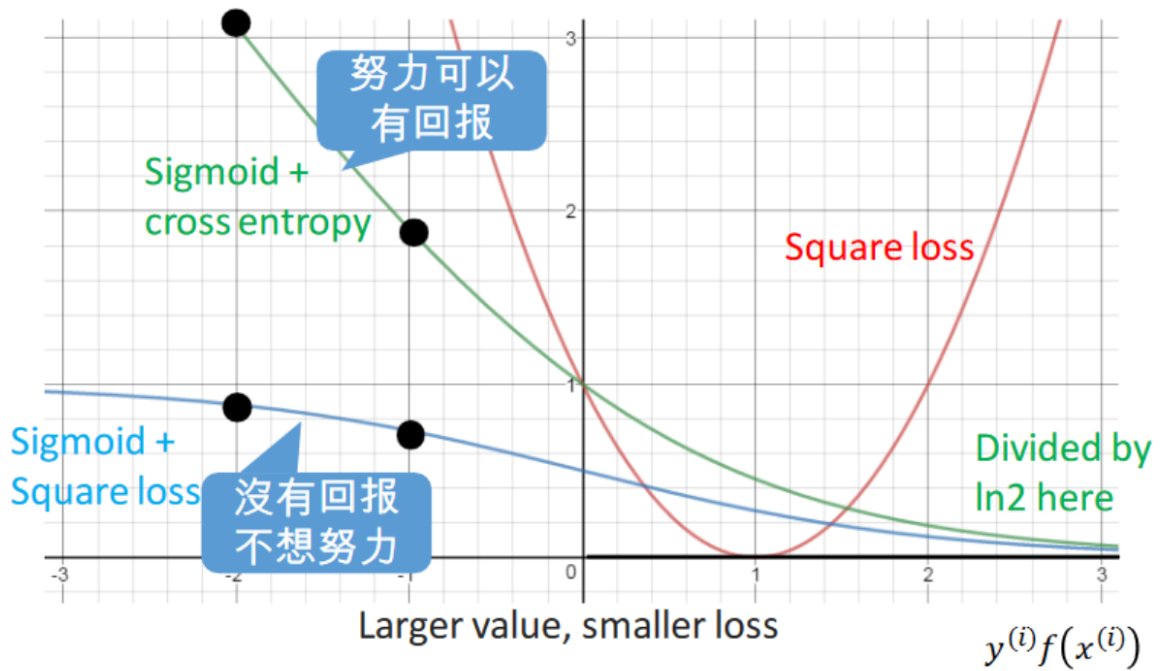
- 初始化拉格朗日乘子向量 α 为0向量，并选择违反KKT条件的样本点作为第一个优化对 (α_i, α_j) 。
- 固定其他样本点的 α ，通过解析方式优化选择的 (α_i, α_j) ，使目标函数有所改善。
- 更新对应的阈值 b 和其他的拉格朗日乘子 α 。
- 重复步骤2和步骤3，直至达到收敛条件。

Hinge损失

Hinge损失 (Hinge Loss): SVM使用的损失函数，用于衡量预测结果和真实标签之间的差距，定义为 $L(y, f(\mathbf{x})) = \max(0, 1 - yf(\mathbf{x}))$ 。其中 y 是样本的真实标签， $f(\mathbf{x})$ 是通过分类器预测的函数值。Hinge损失对于正确分类的样本损失为0，对于误分类的样本，损失随着误分类程度的增加而增加。

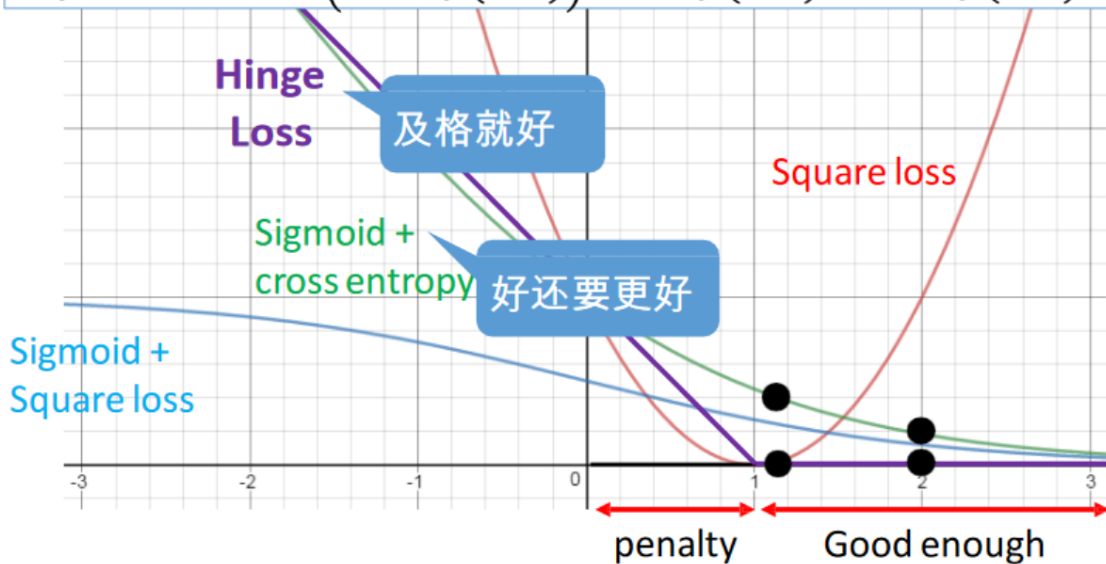
Hinge Loss + Kernel Method = Support Vector Machine (SVM)

损失函数 Loss function



损失函数 Loss function

$$\begin{aligned} \text{If } y^{(i)} = 1, \max(0, 1 - f(x^{(i)})) & \quad 1 - f(x^{(i)}) < 0 \quad f(x^{(i)}) > 1 \\ \text{If } y^{(i)} = -1, \max(0, 1 + f(x^{(i)})) & \quad 1 + f(x^{(i)}) < 0 \quad f(x^{(i)}) < -1 \end{aligned}$$



优缺点

6.1 优点

- 有严格的数学理论支持，可解释性强，不依靠统计方法，从而简化了通常的分类和回归问题；
- 能找出对任务至关重要的关键样本（即：支持向量）；
- 采用核技巧之后，可以处理非线性分类/回归任务；
- 最终决策函数只由少数的支持向量所确定，计算的复杂性取决于支持向量的数目，而不是样本空间的维数，这在某种意义上避免了“维数灾难”。

6.2 缺点

- 训练时间长。当采用 SMO 算法时，由于每次都需要挑选一对参数，因此时间复杂度为 $O(N^2)$ ，其中 N 为训练样本的数量；
- 当采用核技巧时，如果需要存储核矩阵，则空间复杂度为 $O(N^2)$ ；
- 模型预测时，预测时间与支持向量的个数成正比。当支持向量的数量较大时，预测计算复杂度较高。

因此支持向量机目前只适合小批量样本的任务，无法适应百万甚至上亿样本的任务。

SVM算法对大规模训练样本和多分类问题（多个二类支持向量机的组合）难以实施。

思考题

简述一下本节课介绍的SVM的两个最重要的特点，以及带来的好处。

支持向量机（Support Vector Machine, SVM）是一种重要的机器学习算法，具有以下两个最重要的特点，并由此带来了一些好处：

1. 最大间隔分类（Maximal Margin）：SVM的核心思想是通过最大化分类器的间隔来进行分类。它寻找一个超平面，将不同类别的样本点分开，并使得两个类别的支持向量（距离超平面最近的样本点）之间的间隔最大化。这种最大间隔分类的特点使得SVM在处理线性可分问题时能够得到良好的分类效果。

好处：

- 提高泛化能力：最大间隔分类使得SVM对新样本的泛化能力较强，能够更好地适应未见过的数据。
- 抗噪能力强：最大间隔分类能够有效地降低对噪声样本的敏感性，提高对于异常值的鲁棒性。
- 高准确性：SVM通过最大化间隔来找到一个较好的决策边界，使得模型具有较高的泛化能力，能够处理复杂的非线性分类问题。

最大化间隔是SVM的一个重要特点，它通过找到一个决策边界（超平面），使得与两个类别中最近的样本点之间的距离（即间隔）最大化。这种最大化间隔的方法有助于提高模型的泛化能力，减少过拟合的风险。

在数学形式上，对于线性可分的情况，SVM的决策边界由权重向量 w 和偏置项 b 确定，表示为 $w^T x + b = 0$ 。SVM的目标是求解以下优化问题：

$$\min_{w,b} \frac{1}{2} |w|^2$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1, \forall i = 1, 2, \dots, n$$

其中， y_i 是样本 x_i 的标签，取值为 $+1$ 或 -1 ， n 是样本的数量。

2. 核函数的引入：SVM使用核函数（Kernel Function）来将数据映射到高维特征空间中，从而在高维空间中实现非线性分类。通过使用核函数，SVM可以在原始特征空间中进行非线性分类，而无需显式地进行高维特征空间的计算。

好处：

- 处理非线性问题：通过核函数，SVM能够处理原始特征空间中非线性关系的问题，从而拓展了其应用范围。
- 减少特征工程：使用核函数可以避免手动进行复杂的特征变换，减少了特征工程的工作量。

- 解决维度灾难：通过核函数的应用，SVM可以将低维输入空间映射到高维特征空间，从而解决了维度灾难问题。它能够处理高维数据，有效地进行特征提取和分类。

这些特点使得SVM成为一种强大的分类算法，并在许多领域中取得了广泛的应用。它能够处理线性可分和线性不可分问题，并具有良好的泛化能力和鲁棒性。同时，通过核函数的引入，SVM还可以有效地处理非线性分类问题，减少了特征工程的需求，提高了分类的准确性和效率。

八：贝叶斯

贝叶斯定理是概率论中的一种重要理论，也是贝叶斯统计的基础。它主要用于描述两个条件概率之间的关系。贝叶斯定理可以帮助我们基于现有的证据或信息更新我们对未知事件发生概率的预估。

贝叶斯定理的公式如下：

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

其中：

- $P(A|B)$ 表示给定事件 B 发生的情况下，事件 A 发生的条件概率，也被称为后验概率。
- $P(B|A)$ 表示给定事件 A 发生的情况下，事件 B 发生的条件概率，也被称为似然概率。
- $P(A)$ 和 $P(B)$ 是事件 A 和 B 的边缘概率，也可以理解为它们各自发生的概率， $P(A)$ 被称为先验概率。
- 分母 $P(B)$ 可以理解为对所有可能导致 B 发生的情况进行概率求和，也就是全概率公式 $P(B) = \sum_i P(B|A_i)P(A_i)$ ，这也是使得 $P(A|B)$ 成为一个概率分布的标准化常数。

注意：在应用贝叶斯定理时，我们通常已知 $P(B|A)$ ， $P(A)$ 和 $P(B)$ ，然后用这些信息来计算我们感兴趣的后验概率 $P(A|B)$ 。

在机器学习中的贝叶斯分类器，就利用了贝叶斯定理来预测一个实例最可能属于哪个类别。

三门问题

三门问题是一个经典的概率问题，也是贝叶斯定理应用的一个典型例子。在这个问题中，有三扇门，其中一扇门后面有一辆车，其余两扇门后面是山羊。游戏玩家选择一扇门，然后主持人（知道每扇门后是什么）会打开一扇其他的、后面有山羊的门。然后主持人会问游戏玩家是否要改变原来的选择。

假设你选择了门1，然后主持人打开了门2，门后是山羊。我们要计算的是在这些新的信息之后，车在门1和门3后的后验概率。

设 h_i 表示车在门 i 后， D_i 表示主持人打开门 i 。我们可以利用贝叶斯定理计算后验概率：

$$\text{车在门1后的后验概率: } P(h_1|D_2) = \frac{P(D_2|h_1) \cdot P(h_1)}{P(D_2)} = \frac{\frac{1}{2} \cdot \frac{1}{3}}{\frac{1}{2} \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} + 1 \cdot \frac{1}{3}} = \frac{1}{3}$$

$$\text{车在门3后的后验概率: } P(h_3|D_2) = \frac{P(D_2|h_3) \cdot P(h_3)}{P(D_2)} = \frac{1 \cdot \frac{1}{3}}{\frac{1}{2} \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} + 1 \cdot \frac{1}{3}} = \frac{2}{3}$$

这表明，改变选择会增加获胜的概率。

最大后验假设

在许多学习场景中，学习者会考虑一组假设 H ，并且希望找到在给定观测数据 D 的情况下最有可能的假设 $h \in H$ ，这个假设就叫做最大后验假设。通过贝叶斯定理，最大后验假设可以表示为：

$$h_{MAP} = \arg \max_{h \in H} P(h|D) = \arg \max_{h \in H} \frac{P(D|h) \cdot P(h)}{P(D)}$$

在所有的先验概率 $P(h)$ 都相等的情况下，这就变成了：

$$h_{MAP} = \arg \max_{h \in H} P(D|h)$$

也就是说，选择使得数据出现概率最大的假设。

需要注意的是，以上内容仅是对贝叶斯网络和贝叶斯定理的基础应用进行了简单的解释。在实际问题中，贝叶斯网络的建立和使用还需要更深入的理解和学习。

朴素贝叶斯分类器

1. 高斯朴素贝叶斯 (Gaussian NB) :

- 适用于**连续特征（如实数或实数区间）**的分类问题。
- 假设每个特征的值服从高斯分布。
- 公式: $P(x_i|v_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x_i-\mu_k)^2}{2\sigma_k^2}\right)$
- 其中, $P(x_i|v_k)$ 表示在类别 v_k 下特征 x_i 的概率, μ_k 和 σ_k 分别表示类别 v_k 下特征 x_i 的均值和标准差。

2. 多项式朴素贝叶斯 (Multinomial NB) :

- 适用于**多变量离散特征（如词频）**的分类问题。
- 假设特征的值是多项式分布。
- 公式: $P(x_i|v_k) = \frac{N_{x_i,v_k} + \alpha}{N_{v_k} + \alpha \times n}$
- 其中, $P(x_i|v_k)$ 表示在类别 v_k 下特征 x_i 的概率, N_{x_i,v_k} 表示类别 v_k 中特征 x_i 出现的次数, N_{v_k} 表示类别 v_k 的样本数量, α 是平滑参数, n 是特征取值的个数。

3. 伯努利朴素贝叶斯 (Bernoulli NB) :

- 适用于**二元离散特征或非常稀疏的多变量离散特征**的分类问题。
- 假设特征的值是伯努利分布。
- 公式: $P(x_i|v_k) = \frac{N_{x_i,v_k} + \alpha}{N_{v_k} + \alpha \times 2}$
- 其中, $P(x_i|v_k)$ 表示在类别 v_k 下特征 x_i 的概率, N_{x_i,v_k} 表示类别 v_k 中特征 x_i 出现的次数, N_{v_k} 表示类别 v_k 的样本数量, α 是平滑参数, 2 表示特征取值的个数（二元特征）。

贝叶斯网络 (Bayesian Network) 也称为信度网络或有向无环图模型 (directed acyclic graphical model), 是一种用于描述变量之间概率依赖关系的概率图模型。它使用有向无环图 (DAG) 表示变量之间的依赖关系, 并使用条件概率分布 (CPD) 表示变量之间的概率关系。

基础定义:

- 节点: 每个节点代表一个随机变量, 可以是离散的或连续的。
- 边: 边表示随机变量之间的依赖关系。箭头的方向表示依赖的方向, 例如, 箭头从节点A指向节点B, 表示B依赖于A, A是B的父节点, B是A的子节点。
- 条件概率表 (CPT): 每个节点都有一个条件概率表, 描述给定其父节点时, 该节点自身取值的条件概率。

贝叶斯网络具有以下几个性质:

1. 因果性: 如果节点A指向节点B, 那么A就是B的因, B是A的果。
2. 条件独立性: 在网络中, 给定节点A的父节点, 节点A与其非后裔节点条件独立。非后裔节点指的是既不是A的子节点, 也不是A的子节点的子节点。
3. 可解释性: 贝叶斯网络的结构可以表达领域的知识, 每个箭头可以解释为因果关系或影响关系。

贝叶斯网络的联合概率分布可以使用链式法则 (chain rule) 进行计算, 表示为:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | Pa(X_i))$$

这是贝叶斯网络的基本概念。在实际应用中, 贝叶斯网络可用于预测、诊断、推理等任务。

EM算法

期望最大化 (Expectation Maximization, EM) 算法常在以下情况下使用:

- 数据只部分可观察;
- 无监督聚类: 目标值不可观察;
- 监督学习: 某些实例属性不可观察。

EM算法的应用范围包括贝叶斯信念网络的训练、无监督聚类和隐藏马尔科夫模型的学习。

EM问题的一般形式为:

- 已知：可观察数据 $X = x_1, \dots, x_m$ ，不可观察数据 $Z = z_1, \dots, z_m$ ，依赖 X 和 h ，参数化概率分布 $P(Y|h)$ ，其中 $Y = y_1, \dots, y_m$ 为完整数据， $y_i = \langle x_i, z_i \rangle$ ， h 是参数的假设值；
- 确定：（局部）最大化 $E[\ln P(Y|h)]$ 的 h 。

EM方法的定义和步骤包括：

- 定义似然函数 $Q(h'|h)$ ，使用可观察的 X 和当前假设的参数 h 来估计 Z ，计算 $Y = X \cup Z$ ：

$$Q(h'|h) = E[\ln(P(Y|h')|h, X)]$$
- EM算法：E步骤：使用当前假设 h 和可观察数据 X 来估计 Y 的概率分布，计算 $Q(h'|h)$ ；M步骤：用最大化该 Q 函数的假设 h' 替换假设 h ，即 $h = \operatorname{argmax}_{h' \in H} Q(h'|h)$ 。

在估计 k 均值的EM算法中，每个实例 x 由以下步骤生成：

- 随机选择 k 个高斯分布之一；
- 根据该高斯分布生成实例。

给定由 k 个高斯混合生成的 X 的实例，未知的 k 个高斯的均值 $\langle \mu_1, \dots, \mu_k \rangle$ ，我们不知道哪个实例 x_i 是由哪个高斯生成的，目标是确定 $\langle \mu_1, \dots, \mu_k \rangle$ 的最大似然估计。

EM算法是一种迭代技术，设计用于概率模型。EM算法的步骤如下：

- E步骤：计算每个隐藏变量 z_{ij} 的期望值 $E[z_{ij}]$ ，假设当前假设 $h = \langle \mu_1, \mu_2 \rangle$ 成立。
- M步骤：假设每个隐藏变量 z_{ij} 的值是其在E步骤中计算出的期望值 $E[z_{ij}]$ ，计算新的最大似然假设 $h' = \langle \mu'_1, \mu'_2 \rangle$ 。然后用 $h' = \langle \mu'_1, \mu'_2 \rangle$ 替换 $h = \langle \mu_1, \mu_2 \rangle$ 。

这个过程使用以下公式进行计算：

E步骤：计算每个隐藏变量 z_{ij} 的期望值 $E[z_{ij}]$ ，假设当前假设 $h = (\mu_1, \mu_2)$ 成立。

M步骤：假设每个隐藏变量 z_{ij} 的值是其在E步骤中计算出的期望值 $E[z_{ij}]$ ，计算新的最大似然假设 $h' = (\mu'_1, \mu'_2)$ 。然后用 $h' = (\mu'_1, \mu'_2)$ 替换 $h = (\mu_1, \mu_2)$ 。

这个过程使用以下公式进行计算：

$$E[z_{ij}] = \frac{e^{-\frac{(x_i - \mu_1)^2}{2\sigma^2}}}{e^{-\frac{(x_i - \mu_1)^2}{2\sigma^2}} + e^{-\frac{(x_i - \mu_2)^2}{2\sigma^2}}}$$

$$M\text{步骤: } \mu'_j = \frac{\sum_{i=1}^m x_i E[z_{ij}]}{\sum_{i=1}^m E[z_{ij}]}$$

以上是EM算法的步骤和公式。在E步骤中，计算了隐藏变量 z_{ij} 的期望值，然后在M步骤中，根据这些期望值更新了参数 μ_j 。重复执行E步骤和M步骤，直到收敛达到最优参数估计。

贝叶斯信念网络

贝叶斯置信网络(Bayesian Belief Networks, 也称为Bayes nets)，以及贝叶斯网络中的条件独立性、参数学习和结构学习等概念。

- 条件独立性**: 在贝叶斯网络中，条件独立性是关键的概念。如果给定随机变量 Z 的值，随机变量 X 的概率分布独立于随机变量 Y 的值，那么我们称 X 在给定 Z 时条件独立于 Y ，用数学公式表示为 $P(X|Y, Z) = P(X|Z)$ 。这种性质在朴素贝叶斯分类器中被用来简化计算。
- 贝叶斯置信网络**: 贝叶斯置信网络是一种概率图模型，它可以用来描述变量之间的条件依赖性。网络中的每一个节点都代表一个随机变量，而节点之间的边则代表这些变量之间的概率依赖关系。
- 贝叶斯网络的推理**: 给定一些变量的观察值，我们可以使用贝叶斯网络来推断一个或多个其他变量的值（或其概率分布）。这种推断过程可能是NP难的，但在实践中，有许多成功的例子。
- 贝叶斯网络的学习**: 学习贝叶斯网络包括两个主要的部分：参数学习和结构学习。
 - 参数学习**: 参数学习的目标是在给定网络结构和训练数据的情况下，确定网络中每个节点的条件概率分布。常见的方法包括最大似然估计、贝叶斯估计以及在不完全数据下的参数学习等。
 - 结构学习**: 结构学习的目标是确定网络的拓扑结构，即变量之间的依赖关系。常见的方法包括基于搜索和评分的方法以及基于条件独立性测试的方法。

这些理论方法在实践中有着广泛的应用，可以用于概率推理、分类、回归、聚类等诸多任务。

思考题

简述贝叶斯最优分类器、朴素贝叶斯分类器和贝叶斯信念网络的区别？

贝叶斯最优分类器 (Bayes Optimal Classifier) :

贝叶斯最优分类器是一个理论上的概念，它表示如果我们知道整个概率模型，即所有类别的先验概率和所有特征的条件概率，我们就可以构建一个误差最小的分类器。这是基于最小化总体错误率的概念，即对于每一个实例，都选择最高后验概率的类别。然而，真实世界中的问题是，我们通常不知道完整的概率模型，因此必须从数据中估计这些概率。

朴素贝叶斯分类器 (Naive Bayes Classifier) :

朴素贝叶斯分类器是一种简化的贝叶斯分类器，它假设所有的特征都是条件独立的。这个假设使得算法可以在每个类别中独立地估计每个特征的条件概率，从而大大简化了计算。朴素贝叶斯分类器对数据的这种“朴素”假设在许多实际问题中是不成立的，但是在许多情况下，它仍然能够提供相当好的分类性能。

贝叶斯信念网络 (Bayesian Belief Network) :

贝叶斯信念网络或贝叶斯网络是一种概率图模型，它表示变量之间的概率依赖关系。相比于朴素贝叶斯分类器，贝叶斯网络可以表示更复杂的依赖关系，它并不需要假设所有的特征都是条件独立的。因此，如果我们对问题的结构有一些先验知识，我们就可以利用这种知识来构建更好的模型。然而，学习和推理在贝叶斯网络中通常要比在朴素贝叶斯分类器中更为复杂。

总的来说，这三种模型的区别主要在于它们对问题的假设和复杂性：贝叶斯最优分类器假设我们已知所有的概率，朴素贝叶斯分类器假设输入变量之间是条件独立的，贝叶斯信念网络则不需要这些假设，但模型的复杂性会增加。

九：聚类

聚类分析的目标是找到一组对象，这些对象在一组内部相互相似（或相关），并且与其他组中的对象不同（或无关）。优化目标通常是最大化组间距离（Inter-cluster distances）并最小化组内距离（Intra-cluster distances）。

聚类分析的应用领域广泛，包括推荐系统和广告（如聚类用户用于商品/广告推荐，聚类项目用于相关项目建议），文本挖掘（如聚类文档用于相关搜索，聚类单词用于查询建议），图像搜索（如聚类图像用于相似图像搜索和重复检测），语音识别或分离（聚类语音特征）。

聚类的概念有时可能含糊不清，如何定义一个集群可以有不同的解读方式。

好的聚类方法会产生高质量的集群，具有高内部类别相似性和低类间相似性。聚类结果的质量取决于方法使用的相似度测量以及其实现。一个聚类方法的质量还可以通过其发现隐藏模式的能力来衡量。

主要的聚类方法包括：

- 划分方法：构建各种分区，然后通过某种标准评估它们，例如最小化平方误差和。典型方法：k-means, ISODATA, k-medoids, Kernel K-means, CLARANS
- 层次方法：使用某种标准创建数据集的层次分解。典型方法：Agnes, Diana, BIRCH, ROCK, CAMELEON
- 基于密度的方法：基于连通性和密度函数。典型方法：DBSCAN, OPTICS, DenClu
- 基于模型的方法：为每个集群假设一个模型，并试图找到最适合该模型的其他模型。典型方法：EM, SOM, COBWEB
- 基于网格的方法：基于多级粒度结构。典型方法：STING, WaveCluster, CLIQUE
- 基于频繁模式的方法：基于频繁模式的分析。典型方法：pCluster

聚类集合之间的其他区别包括：互斥与非互斥，模糊与非模糊，部分与完整，异质与同质。

聚类类型包括：良好分离的集群，中心基础的集群，相邻的集群，基于密度的集群，属性或概念化的集群。

聚类算法包括分割聚类、层次聚类和基于密度的聚类。

分割聚类(Partitional Clustering)

K-means 聚类是分割聚类的一种方法。每个聚类都与一个质心（中心点）相关联，每个点都被分配到最近的质心的聚类中。聚类的数量，也就是 K 值，需要预先指定。

K-means 的初始质心通常是随机选择的，因此得到的聚类结果会因每次运行而有所不同。质心通常是聚类中点的平均值，'接近'的度量通常用欧氏距离或余弦相似度来衡量。

欧氏距离定义为： $d(x, y) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2}$ ，这里 x 和 y 是两个 n 维的向量。

在文档向量的情况下，计算余弦相似度的公式为：

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{|\mathbf{x}| |\mathbf{y}|}$$

其中， $\mathbf{x}$ 和 $\mathbf{y}$ 是文档向量， $|\mathbf{x}|$ 和 $|\mathbf{y}|$ 是各自向量的模， $\mathbf{x} \cdot \mathbf{y}$ 是向量的点积。

- 欧氏距离：
 - 特点：欧氏距离是常用的距离度量方式，衡量了数据点在每个维度上的差异。它适用于数值型特征，可以对数值间的距离进行直观的度量。
 - 适用情况：当数据的维度较小且数值特征具有明显的物理意义时，欧氏距离是一个合适的选择。例如，在二维平面上的坐标点，可以使用欧氏距离度量它们之间的距离。
- 余弦相似度：
 - 特点：余弦相似度衡量了数据点之间的方向性和夹角的相似程度，而不考虑其绝对数值大小。它适用于文本、稀疏向量等表示的特征，可以忽略维度上的缩放因素，更关注数据的方向性。
 - 适用情况：当数据的特征表示为向量，并且不关心其绝对大小，而更关心它们之间的方向和相似度时，余弦相似度是一个合适的选择。例如，在文本聚类中，使用TF-IDF表示的文档向量可以使用余弦相似度度量它们之间的相似度。

K-means 算法的复杂度为 $O(n * K * I * d)$ ，其中， n 是点的数量， K 是聚类的数量， I 是迭代次数， d 是属性的数量。

K-means 算法的一个主要问题在于初始质心的选择。如果在真实聚类中存在 K 个质心，那么每个聚类选出一个质心的概率是非常小的，特别是当 K 很大时。选择初始质心的策略包括多次运行以找到最佳结果，使用层次聚类对样本进行预处理以确定初始质心，选择超过 K 个的初始质心，然后从这些质心中选择最广泛分隔的 K 个。

评价 K-means 聚类的最常见度量是平方误差和 (SSE)。每个点的误差是它到最近聚类的距离，求得 SSE 的方式是将这些误差平方后求和：

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

其中， x 是聚类 C_i 中的一个数据点， m_i 是聚类 C_i 的代表点（即质心）。

除此之外，K-means 还存在其它问题，例如可能会产生空聚类，对异常值敏感，以及当聚类的大小、密度、形状不同样本分布不均匀时，效果不佳。为了解决这些问题，我们可以在聚类过程中进行预处理（如数据归一化和消除异常值）和后处理（如消除可能代表异常值的小聚类，分裂高 SSE 的“松散”聚类，合并“接近”且 SSE 相对较低的聚类）。

层次聚类 (Hierarchical Clustering)

层次聚类生成一组组织为层次树的嵌套簇，并且可以通过树图 (dendrogram) 进行可视化，该图可以记录合并或分裂的序列。

层次聚类的优点：

- 不需要假设任何特定数量的簇，通过在适当的层次上“剪断”树图，可以获得任何所需数量的簇。
- 它们可能对应有意义的分类法，例如在生物科学中的动物王国，系统发育重建等。

层次聚类的类型：

- **凝聚型 (Agglomerative)**：从将每个点视为单独的簇开始，然后在每一步中合并最接近的一对簇，直到只剩下一个簇（或 k 个簇）。
- **分裂型 (Divisive)**：从一个包含所有点的簇开始，然后在每一步中分裂一个簇，直到每个簇只包含一个点（或者有 k 个簇）。

凝聚型聚类算法 (Agglomerative Clustering Algorithm)：

1. 计算亲近度矩阵 (proximity matrix)。
2. 将每个数据点作为一个簇。
3. 重复以下步骤：
 1. 合并最接近的两个簇。
 2. 更新亲近度矩阵。
4. 直到只剩下一个簇为止。

亲近度的计算是关键操作，定义两个簇之间距离的不同方法区分了不同的算法。

簇间相似度的定义：

- **最小链接 (MIN or Single Link)**：两个簇的相似度基于不同簇中两个最相似（最接近）的点。计算简单，只需要考虑两个簇中距离最近的点的距离作为相似度。最小链接的优点是能够发现具有不规则形状的簇，但它对噪声和离群点非常敏感。选择最小链接适合于希望发现不规则形状簇的情况。
- **最大链接 (MAX)**：两个簇的相似度基于不同簇中两个最不相似（最远）的点。最大链接的优点是能够处理噪声和离群点，但它有时会导致产生比较大的簇。选择最大链接适合于希望将离群点排除在聚类之外的情况。
- **组平均 (Group Average)**：两个簇的接近度是簇内两点之间接近度的平均值。公式为：
$$proximity(Cluster_i, Cluster_j) = \frac{\sum_{p_i \in Cluster_i, p_j \in Cluster_j} proximity(p_i, p_j)}{|Cluster_i| \times |Cluster_j|}。$$
- **Ward的方法 (Ward's Method)**：两个簇的相似度基于当两个簇合并时，平方误差的增加。

层次聚类的时间和空间需求：

- 需要 $O(N^2)$ 的空间，因为它使用亲近度矩阵，其中 N 是点的数量。
- 在许多情况下，需要 $O(N^3)$ 的时间，因为有 N 个步骤，并且在每个步骤中，大小为 N^2 的亲近度矩阵必须被更新和搜索。对于一些方法，复杂性可以降低到 $O(N^2 \log(N))$ 时间。

层次聚类的问题和限制：

- 一旦做出将两个簇合并的决定，就无法撤销。
- 没有直接最小化的目标函数。
- 不同的方案可能有以下一个或多个问题：对噪声和离群值敏感；处理不同大小的簇和凸形状有困难；打破大簇。

基于密度的聚类(Density-based clustering)

基于密度的聚类是一种按照密度（局部聚类准则）进行聚类的方法，比如密度连接的点。每一个聚类都有明显高于聚类外部的点密度。

基于密度的聚类的主要特征包括：

- 能发现任意形状的聚类。
- 能处理噪声。
- 一次扫描就可以进行聚类。
- 有几个著名的研究方法，如 DBSCAN, GDBSCAN, OPTICS, DENCLUE, 和 CLIQUE。

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种基于密度的算法。

- 密度定义为指定半径 (Eps) 内的点的数量。
- 如果一个点在 Eps 内有超过指定数量的点 (MinPts)，那么这个点就是核心点。这些点是聚类的内部点。
- 边界点在 Eps 内有小于 MinPts 的点，但是在核心点的邻域内。
- 噪声点是任何不是核心点或边界点的点。

- 这个算法不需要指定聚类的数量，参数较少。

DBSCAN算法首先去除噪声点，然后对剩余的点进行聚类。

DBSCAN在处理有噪声的数据以及能处理不同形状和大小的聚类时表现良好。但在处理密度不同的聚类和高维数据时表现不佳。

对于确定DBSCAN的EPS和MinPts参数，一种思路是对于一个聚类中的点，其第k个最近邻居的距离大致相同。噪声点的第k个最近邻居的距离会更远。因此，可以画出每个点到其第k个最近邻居的距离的排序图。

对于聚类结果的有效性（Cluster Validity），在监督分类中，我们有多种衡量模型优良性的方法，如准确率，精度，召回率。对于聚类分析，类似的问题是如何评价结果聚类的"好坏"呢？

我们之所以想评价聚类结果，有以下原因：

- 避免在噪声中找到模式。
- 比较不同的聚类算法。
- 比较两组聚类。
- 比较两个聚类。

对于聚类结果的评估，通常需要使用一些度量指标来衡量聚类的质量，例如紧密度、分离度、轮廓系数等。

十：集成学习

集成学习是一种机器学习范式，其中多个学习者（模型）被训练以解决同一问题。与尝试从单个模型获取最优预测不同，集成方法尝试通过在某种程度上结合多个模型的预测来改善预测的准确性或稳健性。

集成学习有两种主要的训练模式：并行和序列。并行方法，如Bagging和Random Forest，训练多个基础学习器，每个学习器的训练是独立的。序列方法，如Boosting和AdaBoost，每个基础学习器的训练依赖于前一个学习器。

基础学习器 (Base Learner)

- **同质**：所有学习者都是同一种类型（例如，决策树，神经网络，支持向量机等）。
- **异质**：学习者可能是不同类型的。

集成学习的示例

构建一个集成预测器，该预测器结合了model1、model2、model3的单独决策。成功的集成需要每个成员都有低错误率，并且做出不同的错误。

集成输入的多样性

- **不同类型的学习者** - DT（决策树），NN（神经网络），KNN（K近邻），SVM（支持向量机）等。
- **不同的训练过程** - 不同的训练集（Bagging中的自举采样，Boosting中的序列采样等），不同的参数（神经网络中的隐藏层神经元数量和初始连接权重等），不同的特性集（随机子空间，随机森林等）。
- **不同的输出表示** - 翻转输出，输出涂抹，ECOC等。
- **混合类型**

集成方法 (Ensemble Methods)

根据个体学习者（通常是弱学习者）的生成模式分类，主要有以下类型：

- **并行方法**：
 - Bagging：通过重复抽样生成训练集，对每个训练集进行训练得到一个预测器，然后通过投票或平均的方式得到最后结果。
 - Random Forest：它是Bagging的一个扩展变体，基础学习器为决策树，同时对样本和特征进行随机采样。
- **序列方法**：

- Boosting: 迭代训练基础学习器, 每一步的训练集依赖前一步的结果, 前一步分错的样本在后面会得到更高的权重、
- GBDT使用Boosting方法, 通过迭代地训练一系列决策树, 每个决策树都尝试纠正前一个树的错误。

Bagging (装袋法) 及其随机森林方法

装袋法 (Bootstrap Aggregating, 简称Bagging) 的主要思想是通过随机采样生成多个训练集, 然后分别对这些训练集训练得到多个模型, 最后通过多数投票或者平均的方法来获得最后的预测结果。

随机森林 (Random Forest)

随机森林是Bagging的一个扩展变体, 基础学习器为决策树。在随机森林中, 除了样本采样带来差异性外, 还引入了特征的随机选择来增加森林中树的多样性, 进一步去相关。

随机森林的算法过程

1. 对原始数据集进行自举采样 (Bootstrap Sampling) 得到N个样本集。
2. 对每一个样本集构建一颗决策树:
 - 选择m个特征子集进行最优分裂点测试。
 - 每棵树都尽可能地生长, 而不进行剪枝。
3. 最终的模型是所有决策树的预测结果的平均值 (回归问题) 或者投票决定 (分类问题) 。

随机森林的优点

1. 使用全部数据进行训练: 随机森林可以使用所有的数据进行训练, 不需要留出数据进行测试。不需要进行传统的交叉验证。
2. OOB评估: 通过使用袋外样本 (out-of-bag samples), 可以对当前树的性能进行评估。这些袋外样本是在构建每棵树时未使用的样本, 可以用于评估模型的准确性。
3. 高预测准确性: 随机森林在预测方面具有高水平的准确性, 能够取得较好的预测结果。同时, 随机森林适用于分类和回归问题。
4. 抗过拟合能力强: 随机森林能够有效地避免过拟合 (overfitting) 的问题, 对于训练数据的噪声和异常值具有一定的鲁棒性。
5. 不需要事先进行特征选择: 随机森林不需要事先进行特征选择, 它能够自动处理高维数据和具有相关性的特征。

随机森林的缺点

1. 难以解释模型的推理过程: 由于随机森林是由多个决策树组成的集合, 模型的推理过程较为复杂, 难以直观地解释每个特征对结果的影响。
2. 训练时间较长: 相对于单个决策树, 随机森林需要构建多个决策树, 并进行投票或平均等操作, 因此训练时间较长。
3. 模型存储空间较大: 由于随机森林包含多个决策树模型, 需要存储每棵决策树的结构和参数, 因此模型的存储空间较大。
4. 对于高维稀疏数据效果较差: 当数据具有高维度和大量稀疏特征时, 随机森林的表现可能不如其他模型效果好, 因为决策树在高维空间中容易失去区分性能。
5. 对异常值敏感: 随机森林对于异常值比较敏感, 异常值可能会对模型的性能产生较大影响。

提升法 (Boosting) 及其相关算法

提升法 (Boosting) 是一种有效的机器学习方法, 其基本思想是将一组相对较弱的分类器结合起来, 形成一个更强大的分类器。Boosting 算法是以序列方式训练分类器, 每个新分类器都根据已训练的分类器的性能进行修改。

Boosting 方法的特性

- 分类器是依次生成的。
- 关注最具信息的数据点。
- 训练样本有权重。
- 输出通过加权投票进行组合。
- 可以创建任意强大的分类器。
- 基础学习器可以任意弱，只要比随机猜测好就行。

自适应增强 (Adaptive Boosting, AdaBoost)

AdaBoost 是一种自适应的 Boosting 算法，它通过调整训练样本的权重，使得在前一轮被错分的样本在后一轮得到更多的关注。

梯度提升决策树 (Gradient Boosting Decision Tree, GBDT)

GBDT 是一种迭代的决策树算法，该算法通过拟合残差来进行学习。

GBDT 的算法过程

- 初始化：训练出一个决策树，使得其最小化损失函数。
- 迭代计算：每一次迭代生成新的模型，新模型通过拟合前一次模型的残差来训练。
- 结合所有的决策树进行预测。

GBDT 的优点

- 可以用于回归和分类问题。
- 强大的预测能力。

分布式梯度增强库 (XGBoost)

XGBoost 是一种优化的分布式梯度提升库，它旨在实现高效，灵活和可移植性。XGBoost 提供了并行的树提升（GBDT，GBM）以解决各种数据科学问题。

轻量级梯度提升机器 (LightGBM)

LightGBM 是一种使用基于树的学习算法的梯度提升框架。LightGBM 旨在分布式和高效，具有以下优点：

- 训练速度快，效率高。
- 内存使用少。
- 更好的精度。
- 支持并行，分布式，以及GPU学习。
- 能够处理大规模数据。

其中，LightGBM 还引入了基于直方图的方法、梯度单边采样 (GOSS) 和互斥特征绑定 (EFB) 来进一步优化速度和内存使用。此外，LightGBM 使用了一种叶子优先的树生长策略，可以更好地处理高维数据，从而提高预测精度。

思考题

简述 Bagging，Boosting 两种集成学习的特点，对比 RF 和 GBDT 算法

Bagging 和 Boosting 是两种常见的集成学习方法。

Bagging（自助聚集法）的特点如下：

1. 采样：Bagging 使用自助采样法（bootstrap sampling），通过有放回地从原始训练集中随机采样生成多个子训练集。
2. 并行训练：每个子训练集使用相同的基本模型独立地进行训练，可以并行处理。
3. 预测：最终的预测结果由所有子模型的平均或投票决定。
4. 鲁棒性：Bagging 对于噪声和过拟合具有一定的鲁棒性，能够减少方差，提高模型的稳定性。

Boosting（提升法）的特点如下：

1. 逐步训练: Boosting通过迭代地训练弱学习器(如决策树), 每个弱学习器都试图修正前一个学习器的错误。
2. 加权样本: 每个样本在训练过程中会被赋予不同的权重, 被错误分类的样本会得到更高的权重, 以便在下一轮迭代中更关注这些样本。
3. 强学习器: 通过加权组合多个弱学习器, 得到一个更强大的集成模型。
4. 自适应调整: Boosting算法通过适应性地调整样本权重和学习器权重, 使得模型更关注困难样本, 能够减少偏差, 提高模型的准确性。

Random Forest (随机森林) 和Gradient Boosting Decision Tree (梯度提升决策树, GBDT) 是集成学习中常用的算法。

Random Forest (RF) 的特点:

1. Bagging策略: RF使用Bagging方法生成多个决策树模型, 每个模型都是在随机子集上独立训练的。
2. 特征随机性: 在每个决策树的节点上, RF随机选择一部分特征进行划分, 减少了模型的相关性。
3. 预测: 通过投票或平均每个决策树的预测结果来进行最终的预测。

Gradient Boosting Decision Tree (GBDT) 的特点:

1. Boosting策略: GBDT使用Boosting方法, 通过迭代地训练一系列决策树, 每个决策树都尝试纠正前一个树的错误。
2. 残差学习: 每个决策树的训练目标是学习前一轮迭代的残差, 以逐步减少整体模型的残差。
3. 加权组合: 通过加权组合所有决策树的预测结果, 得到最终的集成模型。

对比RF和GBDT算法:

1. RF和GBDT都是基于决策树的集成学习方法, 但它们采用了不同的集成策略(Bagging和Boosting)。
2. RF通过随机特征选择和Bagging策略, 减少了模型的相关性和过拟合风险。GBDT通过迭代训练和残差学习, 逐步提高模型的准确性。
3. RF的每个决策树是独立地进行训练, 可以并行处理。而GBDT的决策树是依次训练的, 需要串行处理。
4. RF的预测结果由多个决策树的投票或平均得到。GBDT通过加权组合决策树的预测结果。
5. RF更适用于高维数据和大规模数据集, 而GBDT更适用于解决回归和分类问题, 尤其在处理非线性问题和处理稀疏数据时效果较好。

综上所述, Bagging和Boosting是两种不同的集成学习策略。RF通过Bagging策略和随机特征选择减少模型相关性和过拟合风险, 而GBDT通过Boosting策略和残差学习提高模型的准确性。

十一: 降维

维度灾难 (Curse of Dimensionality)

当处理高维数据时, 即特征数量很大时, 会遇到所谓的“维度灾难”。维度灾难描述的是随着特征空间维度增加, 数据的表示和计算难度急剧增加。例如, 假设我们有一个维度为1的空间, 只需要4个样本就能达到一定的覆盖率。但是如果维度提高到10, 我们需要的样本数量就会上升到 $4^{10} = 1048576$ 。这就是“维度灾难”。

噪声影响 (Noise Influence)

在高维空间中, 即使噪声只有10%, 噪声距离也可能足以掩盖正负样本之间的本质差异。因此, 高维度数据处理中噪声控制非常重要。

降维与特征选择 (Dimensionality Reduction & Feature Selection)

在面对高维数据时, 我们通常需要进行降维和特征选择。降维是通过创建新特征的组合来减少原始特征的数量, 而特征选择则是从所有特征中选择一个子集。这两种方法都是为了减少数据的维度, 简化模型的复杂度, 并提高计算效率。

常见的降维方法有主成分分析 (PCA)、多维缩放 (MDS)、局部线性嵌入 (LLE)、拉普拉斯特征映射 (LE)、等距映射 (Isomap)、t-分布随机近邻嵌入 (t-SNE)、自动编码器、非负矩阵分解 (NMF)、典型相关分析 (CCA)、独立成分分析 (ICA)、概率PCA、核PCA、线性判别分析 (LDA, 有监督) 等。

主成分分析 (PCA)

在众多的降维方法中，主成分分析（PCA）是一种常用的方法。它通过找到最大化方差的方向来选择新的特征，并且这些新的特征是互相正交的。新特征，也称为主成分，通常比原始特征少。具体来说，PCA的算法过程是这样的：

1. 计算数据的协方差矩阵。
2. 计算协方差矩阵的特征值和特征向量。
3. 将特征值从大到小排序，选择前k个最大的特征值对应的特征向量组成一个矩阵P。
4. 用矩阵P将原始数据进行变换，得到新的降维后的数据。

主成分分析（Principal Component Analysis, PCA）是一种常用的无监督学习算法，用于数据的降维处理。主成分分析的目标是将高维度的数据投影到低维度的子空间中，同时尽可能地保留数据的原始信息。

1. 基本原理：

主成分分析的基本思想是找到几个正交的投影方向（主成分），使得投影后的数据方差最大。这些方向即为数据的主要特征向量，方差即为这些特征的重要性。它通过找到最大化方差的方向来选择新的特征，并且这些新的特征是互相正交的。新特征，也称为主成分，通常比原始特征少。

首先计算数据矩阵 X 的协方差矩阵：

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^i - \mu)(x^i - \mu)^T$$

其中，

- Σ 表示协方差矩阵。
- m 是样本数量。
- x^i 表示第 i 个样本向量。
- μ 是数据矩阵 X 的均值向量。

然后对协方差矩阵进行特征值分解，得到特征值和特征向量。选取最大的 k 个特征值对应的特征向量构成的矩阵 W ，将原数据矩阵 X 通过 W 进行线性变换，就得到了降维后的数据。

2. 算法步骤：

主成分分析算法包含以下几个步骤：

1. 对原始数据进行中心化处理（即减去均值）。
2. 计算数据的协方差矩阵。
3. 对协方差矩阵进行特征值分解，得到特征值和特征向量。
4. 选择最大的 k 个特征值对应的特征向量，构成投影矩阵 W 。
5. 用 W 对原数据进行线性变换，得到降维后的数据。

3. 应用：

主成分分析在各种领域都有广泛的应用，例如：图像压缩、噪声过滤、数据可视化等。

4. 局限性：

尽管主成分分析是一种非常实用的降维工具，但它也存在一些局限性。例如，它只能找到线性的主成分，对于非线性的数据结构可能不适用。同时，主成分分析假设数据的主要变化方向是最重要的，但在某些情况下，这可能并不正确。

主成分分析（PCA）虽然是一个强大的降维工具，但它只能发现数据的线性结构。对于复杂的非线性结构，PCA可能无法找到有效的低维表示。这就是流形学习、t-SNE和自编码器等方法的优势所在。

1. 流形学习 (Manifold Learning)：

流形学习是一种用于探索高维数据内在几何结构的技术。它的基本假设是数据虽然存在于高维空间中，但其所有样本都分布在一个远低于原始空间的维度的流形上。流形学习的目标是找到这个流形的参数表示，从而实现高维数据的降维。

2. t-分布领域嵌入算法 (t-SNE)：

t-SNE是一种用于降维的非线性技术，特别适用于将高维数据嵌入到二维或三维空间以进行可视化。t-SNE的基本思想是在高维和低维空间中都计算数据点的配对概率，然后通过最小化这两个配对概率分布之间的KL散度，找到数据的低维表示。

3. 自编码器 (Autoencoder)：

自编码器是一种无监督的神经网络模型，被设计用来有效地编码输入信息。自编码器由两部分组成：编码器和解码器。编码器负责将输入数据压缩到一个“隐藏层”，这一层通常比输入数据的维度要小，而解码器则负责从隐藏层恢复到输入数据的原始维度。自编码器的目标是 최소화 重构误差。

4. 深度自编码器 (Deep Autoencoder) :

深度自编码器是自编码器的一个扩展，它使用多个隐藏层进行编码和解码。这允许模型学习更复杂的数据表示，提高了降维的性能。深度自编码器可以被用于许多任务，如图像重构、文本检索等。

降维是一种常用的机器学习和数据分析手段，其主要目的是减少数据集的特征数量，同时尽量保留原始数据的信息。降维技术主要包括主成分分析 (PCA)、独立成分分析 (ICA) 和特征选择等。

1. 独立成分分析 (ICA) ICA是一种计算方法，用于从多源信号或数据集中分离独立的非高斯随机变量。这种方法通常用于分离混合信号，如鸡尾酒会问题，其中一组人同时进行对话，并且我们想要从混合的语音中分离出单个人的语音。根据中心极限定理，只有独立的非高斯分布的随机变量的混合才会更接近高斯分布。因此，我们可以通过寻找非高斯分布的源信号来分离混合信号。其基本公式是 $x(t) = As(t)$ ，其中 A 是混合矩阵， $s(t)$ 是源信号， $x(t)$ 是观察到的混合信号。通过FastICA算法可以求解出 A 和 $s(t)$ 。

ICA的核心原理是**非高斯性**，也就是在某个方向上观察到的数据的概率分布与高斯分布相比有更大的峰度 (Kurtosis) 或者更重的尾部。这个原理的直观理解是，当我们观察到的数据是多个独立源混合的结果时，混合数据的分布趋向于高斯分布 (根据中心极限定理)。所以非高斯性的方向可能对应着原始的独立源。

ICA的求解过程通常可以使用**FastICA算法**，它是一种迭代的固定点算法，能够寻找最大化非高斯性的投影方向。

2. 主成分分析 (PCA) PCA是一种在数据科学中常见的降维技术，主要用于减小数据的维度，同时保留主要的数据变化方向。PCA通过线性变换将原始数据转化为一组线性不相关的表示，这些表示按照方差的大小排序，方差最大的表示视为数据中最主要的结构，这就是主成分。这种方法常用于高维数据的可视化。

3. 特征选择 特征选择主要有三种方法，包括过滤法 (Filter Method)、包裹法 (Wrapper Method) 和嵌入法 (Embedded Methods)。

- 过滤法：基于特征本身的分布和特征与目标变量之间的关系来选择特征，与学习器无关，计算快速。常见的方法有相关系数、信息增益等。
- 包裹法：根据目标函数 (通常是预测错误率或者AUC) 来选择特征，这意味着必须将特征选择过程与学习器结合起来进行，计算成本相对较高。
- 嵌入法：特征选择与学习器训练过程结合在一起，例如LASSO回归就是一种常见的嵌入法。

需要注意的是，降维和特征选择在很多场景下都是非常有用的，但是它们也可能导致一些信息的丢失。因此，在进行降维和特征选择时，应当充分考虑其可能带来的信息损失和计算成本的权衡。

特征选择是机器学习中的一个重要步骤，通过选取对目标变量影响最大的特征，可以提高模型的性能并减少计算成本。以下是特征选择的三种主要方法的详细介绍：

过滤法 (Filter Method)

过滤法是一种独立于学习器的特征选择方法，它会根据各个特征的重要性进行排序，并选择出最重要的特征。特征的重要性评价主要依赖于特征与目标变量之间的相关性，如皮尔逊相关系数、互信息等。过滤法的计算速度快，但可能会忽视特征之间的相互作用。

包裹法 (Wrapper Method)

包裹法通过一个目标函数 (如预测性能评价指标) 来评估特征的重要性。包裹法会选择一些特征子集，并利用学习算法对特征子集进行训练和验证，从而确定该特征子集的质量。典型的包裹法有向前选择、向后消除等。包裹法通常比过滤法要慢，但由于直接依赖于学习器的性能，所以一般而言能得到更好的性能。

嵌入法 (Embedded Method)

嵌入法在模型的训练过程中进行特征选择，通常会在模型的损失函数中加入正则项来限制模型的复杂度，进而实现特征的自动选择。嵌入法主要包括Lasso、Ridge等线性模型，以及树模型如随机森林和梯度提升树等。嵌入法不仅能考虑特征与目标的相关性，还能考虑特征之间的相互作用，因此通常能获得更好的性能，但计算量较大。

总的来说，选择哪种特征选择方法需要考虑问题的具体需求，例如数据的维度、特征的相关性、学习器的选择、计算资源等因素。